Wonderland Programming Realms

Project Report submitted in the partial fulfilment

of

B.Tech

In

CSEDS 311

by

Srishti L001

Akshada Jangam L025

Under the supervision of

Prof. Nilima Agarwal

(Professor, Visiting, MPSTME)

SVKM's NMIMS University

(Deemed-to-be University)

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (MPSTME)

Vile Parle (W), Mumbai-56

CHAPTER 1: ABSTRACT

"Wonderland Programming Realms" is a desktop quiz app that gamifies learning programming principles into an interactive adventure! The app is developed in Python and utilizes Tkinter as its GUI framework and adds a gamified approach for learning the same material as a regular class.

Students start by creating or selecting a personalized adventurer profile to jump into the activities. There is a built-in data layer in JSON to keep track of their scores, the realms they chose to play, and their overall gameplay history, enabling them to pick up playing where they left off. The main site has four themed "Realms," each related to a general core technology area:

- The Python Rabbit Hole - the basics of Python, including operators, data types, and list comprehensions.
- The C++ Looking Glass - Closer to C++ sort of concepts, including memory management ahead of function overloading.
- The Mad Tea Party - web development learning HTML, CSS, and JavaScript.
- The Queen's Croquet Ground - data science learning through NumPy and Pandas.

Each realm has its own colour, different characters, and stories to create an immersive experience. Each realm contains multiple choice content that is geared toward testing understanding instead of factual recall. Each time the user answers a question correctly, it provides explanation content for the user, promoting learning on each attempt.

The Tkinter GUI is at the heart of the experience, incorporating animated titles, hovers, and a streamlined visual hierarchy. Multiple managed windows (by Toplevel widgets) help the user smoothly navigate from profiles to quizzes.

Content

Chapter No.	Topics
1	Abstract
2	Introduction
3	Concepts used
4	Algorithms
5	Flow chart
6	Code written in Python
7	Output screenshots of all functions
8	Scope of improvements [plans to improve it]
9	References/List of research papers, books, white papers referred. (in IEEE format)

CHAPTER 2: INTRODUCTION

The digital age has brought an unparalleled demand for programming literacy, making computer science education a necessary element of modern education. The first steps into programming can, however, be daunting and frankly disheartening as students get bogged down by theoretical complexities and abstract syntax. Often, traditional approaches to teaching through static input textbooks or revisional online exercises simply don't have the appeal to keep students engaged and do little to connect the abstract theory to applied learning.

To bridge this gap we have developed "Wonderland Programming Realms" - an engaging quiz-based application that turns the teaching of computer science into a fully immersive adventure. Utilizing narrative design and gamification elements inspired by Alice in Wonderland, students will journey through themed “realms” centered on Python, C++, Web Development, and Data Science. The basis of the project is the philosophy that true learning is optimal when users are both cognitively focused and emotionally in tune with that learning. The application includes:

- Persistent Profile System: Each user has a profile where they can track their motivation, scores, and achievements across multiple sessions.
- Themed Visual Interface: The application is presented in Python's Tkinter library with accompanying colourful designs, characters and quotes each geared toward specific realms.
- Challenge-based Learning: Multiple-choice quizzes are conceptual based where real-time feedback is immediate and meaningful.
- Gamification Feedback Loop: Data feedback loops motivate learners to continue their journey through progress markers, scoring and completion badges alike.

In this report, we survey the entire design and build of Wonderland Programming Realms, including its system design, profile handling and GUI layout and behavior. The report ends with a discussion of the project and how well it was able to develop a substantive, engaging learning environment and reflects on what the future could hold for it. The project takes an innovative approach to teaching programming, by combining the elements of storytelling along with a well-structured pedagogical design. The project aims to be a flexible and fun means of teaching programming to beginners.

CHAPTER 3: Concept Implementations

The creation of the application "Wonderland Programming Realms" was a comprehensive practical demonstration of both foundational software engineering principles and advanced Python programming concepts. The application architecture is anchored in Object-Oriented

Programming (OOP), which allows code to be modular, organized, and reusable. For instance, the ProfileManager class encapsulates all the operations related to data management—creating new profiles, saving and loading data, updating user statistics, and generating analytics reports—clearly demonstrating encapsulation and data abstraction. Similarly, ProfileSelectScreen and Alice4App manage the state and behavior of the graphical interface, ensuring the logic for user interaction is distinct from the logic that handles data.

File handling forms the backbone of persistent storage in this project. The program uses JSON files to store user profile information, including usernames, total scores, and completed realms, through serialization (json.dump) and deserialization (json.load). The ProfileManager class reads and writes to wonderland_profiles.json to maintain user data even after the application is closed, ensuring a continuous and consistent user experience. In addition, CSV files are used to maintain structured analytical data. Using Pandas DataFrames, files like player_data.csv and game_stats.csv are created, read, and updated automatically. This approach demonstrates practical use of data analytics libraries—pandas and numpy—to handle tabular data efficiently. Data exported as CSV and text reports allows easy visualization, sorting, and external analysis. Moreover, functions such as export_analytics_report() use file I/O to generate text-based analytics summaries, demonstrating formatted output and file writing.

The inclusion of Tkinter represents the application's event-driven programming model. The GUI employs widgets like Frame, Label, Button, and Listbox, while layout managers (pack and grid) ensure a visually organized interface. Toplevel windows are used for modal dialogs such as profile creation or gameplay screens, exemplifying the management of multiple application states and user focus. The interactive behavior—binding button clicks and actions to event handlers—illustrates the event-driven paradigm where program flow depends on user actions.

Beyond Tkinter's structural capabilities, the application also integrates Matplotlib, preparing the system for visual analytics by embedding graphs directly into the GUI using FigureCanvasTkAgg. This shows a forward-looking approach to integrating data visualization within a Python desktop environment, enabling real-time graphical representation of player statistics. Similarly, NumPy and Pandas provide numerical and analytical capabilities, powering features like ranking top players and aggregating gameplay statistics.

Error handling and exception management are built into the application through the use of try, except, and finally blocks, particularly in file I/O operations. This ensures robustness against issues like missing files or invalid data formats, maintaining application stability. The program also features demonstrations of recursion, lambda functions, and regular expressions within its "Python Spellbook" section, turning conceptual Python topics into interactive learning modules.

Overall, *Wonderland Programming Realms* serves as a cohesive synthesis of multiple Python programming concepts—OOP, file handling, GUI development, data analytics, exception handling, and event-driven design—integrated into a single, stateful application. It effectively

separates data management, business logic, and presentation, embodying professional software engineering structure while demonstrating nearly every practical concept from the Python programming syllabus.

CHAPTER 4: ALGORITHM

This chapter explores the algorithmic structure behind the game's core functions, providing a step-by-step breakdown of the logic that powers gameplay and learning tasks. It highlights how each algorithm supports smooth execution, user interaction, and concept delivery.

Step 1:
Start the application.

Step 2:
Load all existing player profiles from local storage using ProfileManager.

Step 3:
Check if any profiles exist.

- If no profiles exist:
  Prompt user to create a new profile using ProfileSelectScreen.

- If profiles exist:
  Display all available profiles on the ProfileSelectScreen

Step 4:
Wait for the user to select or create a profile.

- If a user creates a new one, save it and reload the list.

- If the user selects an existing one, proceed.

Step 5:
Initialize the main game interface (Alice4App) with the selected profile.
Display the main menu (the "Realms Map").

Step 6:
Wait for a user action from the main menu.
Possible actions:

1. Select a Realm – to start a challenge.

2. View Stats / Settings – to view session info and player stats.

3. Export Analytics – to generate a detailed analytics report.

4. Logout – to return to the Profile Select Screen.

Step 7 (If Realm selected):
Display Realm Info and allow the player to choose a difficulty level (Easy / Medium / Hard).
Load corresponding quiz questions.

Step 8:
Start the quiz session.
For each question:

1. Display the question and options.

2. Accept the player's answer.

3. Check if the answer is correct.

4. Update the temporary realm score accordingly.

Step 9:
After all questions are answered:
Calculate the final realm score.
Update the player's profile data in ProfileManager (including score, completed realms, and stats).
Save updates to both JSON and CSV files.
Check if all realms are completed.

Step 10:
If all realms are completed, show the "Grand Victory" message.
Otherwise, return to the main menu.

Step 11:
If the user selects *Export Analytics*, generate an analytics report text file using ProfileManager.export_analytics_report().

Step 12:
If the user selects *View Stats*, open the settings window showing playtime, scores, and favorite levels.

Step 13:
If the user selects *Logout*, save the current session data, update CSVs, and return to the Profile Select Screen.

Step 14:
When the user closes the app or logs out finally, record session duration, last logout time, and safely exit.

Step 15:
End the program

CHAPTER 5: FLOWCHART

This chapter explores the flowchart of the game, outlining the step-by-step logic and decision-making processes that guide both gameplay and learning activities. It visually represents how players navigate through challenges while interacting with Python concepts in a structured and intuitive manner.

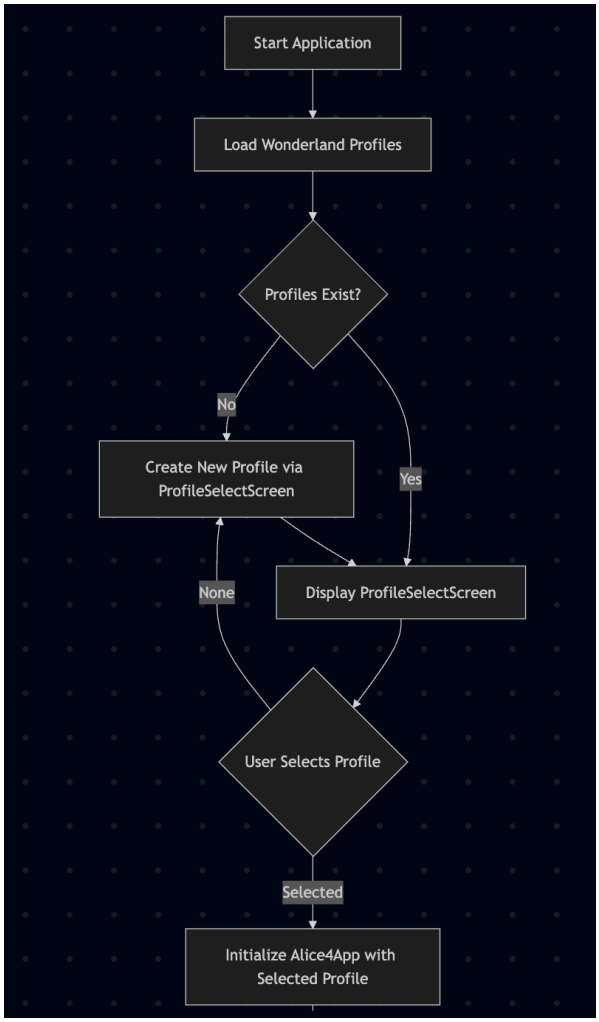

Fig. 5.1. Flowchart

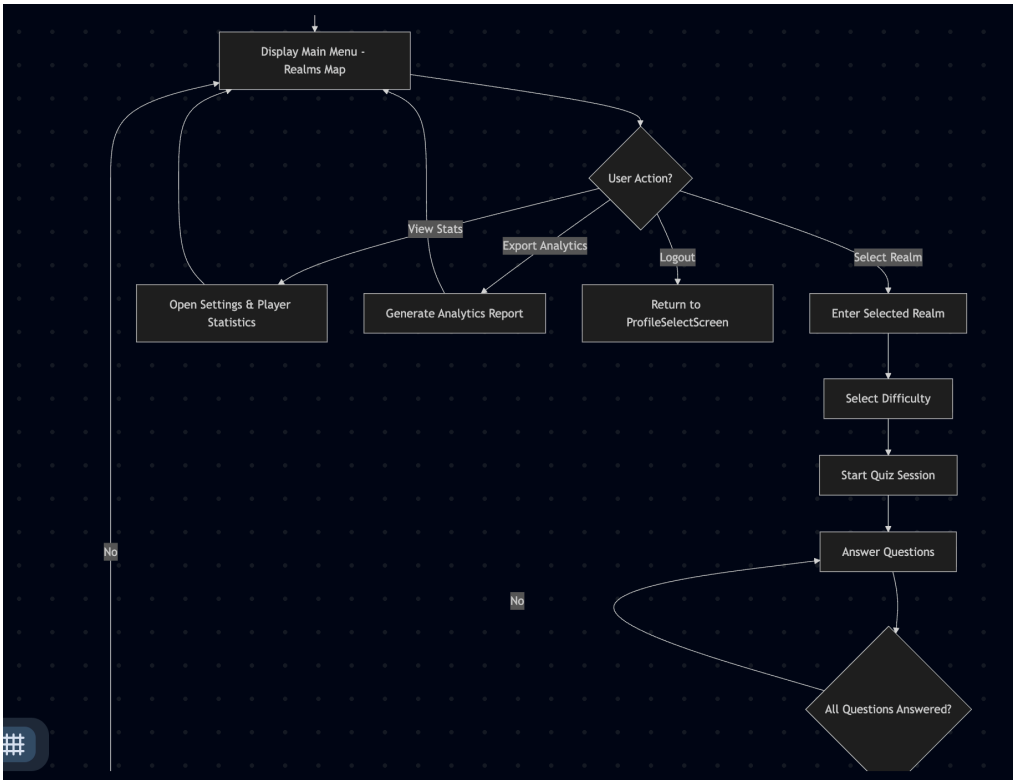

Fig. 5.2. Flowchart

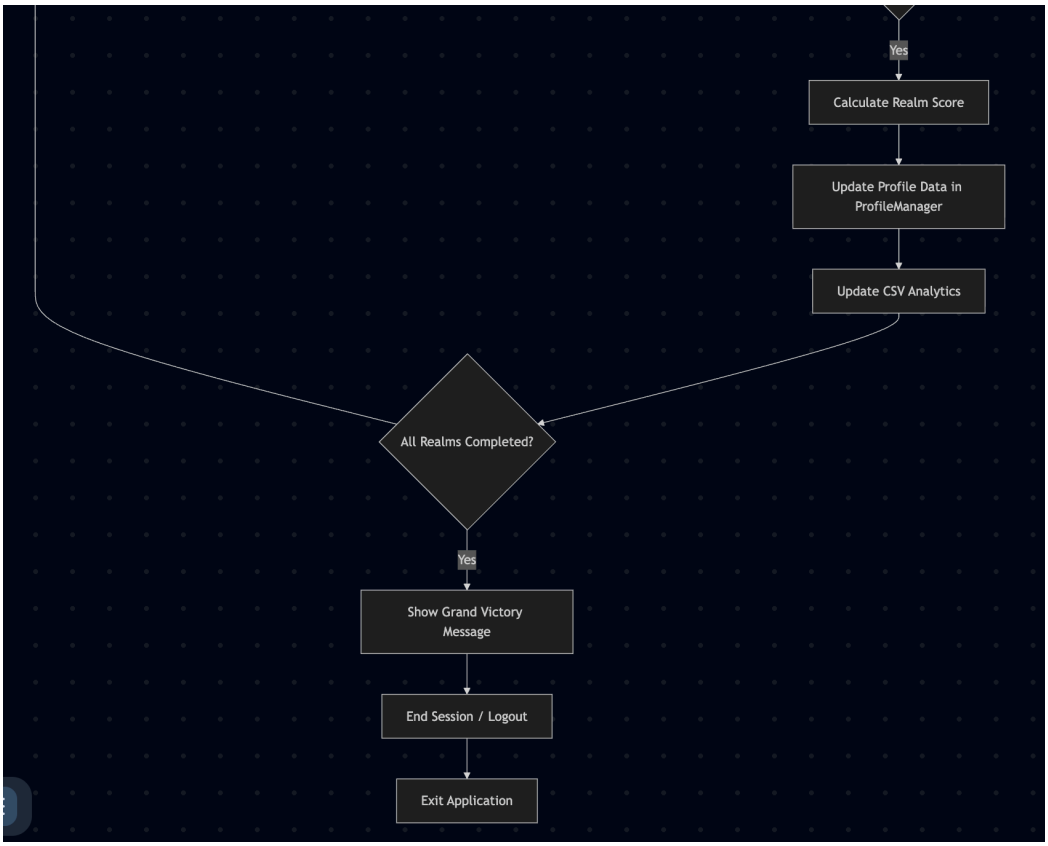

Fig. 5.3. Flowchart

CHAPTER 6: Code written in Python

```
 import os
os.environ['TK_SILENCE_DEPRECATION'] = '1'
import tkinter as tk
from tkinter import ttk, messagebox, scrolledtext
import random
import json
from datetime import datetime
import re
from functools import reduce
from math import factorial

try:
    import numpy as np
    import pandas as pd
    NUMPY_AVAILABLE = True
    PANDAS_AVAILABLE = True
except ImportError:
    NUMPY_AVAILABLE = False
    PANDAS_AVAILABLE = False

try:
    import matplotlib.pyplot as plt
    from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
    MATPLOTLIB_AVAILABLE = True
except ImportError:
    MATPLOTLIB_AVAILABLE = False

PILLOW_AVAILABLE = False
try:
    from PIL import Image, ImageTk
    PILLOW_AVAILABLE = True
except ImportError:
    pass

def create_gradient_frame(parent, color1, color2):
    frame = tk.Frame(parent, bg=color1)
    return frame

def style_button(button, bg_color='#667eea', fg_color='white', hover_color='#764ba2'):
    button.config(
        bg=bg_color,
        fg=fg_color,
        relief='flat',
```

```
        padx=20,
        pady=10,
        font=('Segoe UI', 11, 'bold'),
        cursor='hand2',
        borderwidth=0,
        highlightthickness=0
    )
    def on_enter(e):
        button.config(bg=hover_color)
    def on_leave(e):
        button.config(bg=bg_color)
    button.bind("<Enter>", on_enter)
    button.bind("<Leave>", on_leave)
    return button

def create_styled_label(parent, text, font_size=12, fg='#2d3436', bg='#f8f9fa', bold=False):
    font_weight = 'bold' if bold else 'normal'
    return tk.Label(
        parent,
        text=text,
        font=('Segoe UI', font_size, font_weight),
        fg=fg,
        bg=bg,
        padx=10,
        pady=5
    )

def create_card_frame(parent, bg='#ffffff', padding=20):
    frame = tk.Frame(parent, bg=bg, relief='flat', borderwidth=0)
    frame.pack_propagate(False)
    return frame

class ProfileManager:
    def __init__(self):
            home_dir = os.path.expanduser("~")
            if os.name == "nt":
                base_dir = os.path.join(os.getenv("APPDATA") or home_dir, "Wonderland")
            else:
                base_dir = os.path.join(home_dir, ".wonderland_data")
            os.makedirs(base_dir, exist_ok=True)
            self.base_dir = base_dir
            self.profiles_file = os.path.join(self.base_dir, "wonderland_profiles.json")
            self.player_csv = os.path.join(self.base_dir, "player_data.csv")
            self.game_stats_csv = os.path.join(self.base_dir, "game_stats.csv")
```

```
        self.profiles = self.load_profiles()
        if PANDAS_AVAILABLE:
            self._initialize_csv_files()
            self._sync_json_to_csv()
        else:
            print("⚠️ Pandas not available. CSV analytics features are disabled.")

    def load_profiles(self) -> dict:
        if os.path.exists(self.profiles_file):
            try:
                with open(self.profiles_file, 'r', encoding='utf-8') as f:
                    profiles = json.load(f)
                    for name, profile in profiles.items():
                        profiles[name] = self._ensure_profile_fields(profile)
                    return profiles
            except Exception as e:
                print(f"Error loading profiles: {e}")
                return {}
        return {}

    def _ensure_profile_fields(self, profile: dict) -> dict:
        defaults = {
            "name": profile.get("name", "Unknown"),
            "score": profile.get("score", 0),
            "completed_realms": profile.get("completed_realms", []),
            "realm_scores": profile.get("realm_scores", {}),
            "created_date": profile.get("created_date", datetime.now().strftime("%Y-%m-%d %H:%M:%S")),
            "last_played": profile.get("last_played", datetime.now().strftime("%Y-%m-%d %H:%M:%S")),
            "total_playtime": profile.get("total_playtime", 0),
            "realms_played": profile.get("realms_played", []),
            "realm_play_counts": profile.get("realm_play_counts", {}),
            "realm_difficulty_stats": profile.get("realm_difficulty_stats", {}),
            "last_replay_difference": profile.get("last_replay_difference", 0),
            "favorite_realm_level": profile.get("favorite_realm_level", "N/A"),
            "last_logout": profile.get("last_logout", "N/A")
        }
        for key, value in defaults.items():
            if key not in profile:
                profile[key] = value
        return profile

    def save_profiles(self) -> bool:
        try:
            with open(self.profiles_file, 'w', encoding='utf-8') as f:
```

```
            json.dump(self.profiles, f, indent=4)
        return True
    except Exception as e:
        print(f"Error saving profiles: {e}")
        return False

def _initialize_csv_files(self):
    try:
        if not os.path.exists(self.player_csv):
            player_df = pd.DataFrame(columns=[
                'username', 'total_score', 'realms_completed', 'last_played',
                'created_date', 'realms_played', 'favorite_realm_level',
                'last_replay_difference', 'realm_play_counts', 'realm_difficulty_stats'
            ])
            player_df.to_csv(self.player_csv, index=False)
            print(f"✅ Created {self.player_csv}")
        if not os.path.exists(self.game_stats_csv):
            stats_df = pd.DataFrame(columns=[
                'realm_name', 'average_score', 'attempts_before_full_score',
                'relative_difficulty', 'number_of_players_who_played_realm'
            ])
            stats_df.to_csv(self.game_stats_csv, index=False)
            print(f"✅ Created {self.game_stats_csv}")
    except Exception as e:
        print(f"Error initializing CSV files: {e}")

def _sync_json_to_csv(self):
    if not PANDAS_AVAILABLE:
        return
    for name in list(self.profiles.keys()):
        self._update_player_csv(name)

def _update_player_csv(self, username: str):
    if not PANDAS_AVAILABLE or username not in self.profiles:
        return
    profile = self.profiles[username]
    try:
        df = pd.read_csv(self.player_csv, dtype=str)
    except Exception:
        df = pd.DataFrame()
    row_data = {
        'username': username,
        'total_score': profile.get('score', 0),
        'realms_completed': len(profile.get('completed_realms', [])),
```

```
        'last_played': profile.get('last_played', ''),
        'created_date': profile.get('created_date', ''),
        'realms_played': ';'.join(profile.get('realms_played', [])),
        'favorite_realm_level': profile.get('favorite_realm_level', 'N/A'),
        'last_replay_difference': profile.get('last_replay_difference', 0),
        'realm_play_counts': json.dumps(profile.get('realm_play_counts', {})),
        'realm_difficulty_stats': json.dumps(profile.get('realm_difficulty_stats', {}))
    }
    required_cols = ['username', 'total_score', 'realms_completed', 'last_played', 'created_date',
                     'realms_played', 'favorite_realm_level', 'last_replay_difference',
                     'realm_play_counts', 'realm_difficulty_stats']
    if df.empty:
        df = pd.DataFrame(columns=required_cols)
    for col in required_cols:
        if col not in df.columns:
            df[col] = ''
    safe_values = {k: ('' if v is None else str(v)) for k, v in row_data.items()}
    if username in df['username'].values:
        for k, v in safe_values.items():
            df.loc[df['username'] == username, k] = v
    else:
        new_row_df = pd.DataFrame([safe_values])
        new_row_df = new_row_df.reindex(columns=df.columns, fill_value='')
        df = pd.concat([df, new_row_df], ignore_index=True)
    for col in ['total_score', 'realms_completed', 'last_replay_difference']:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0).astype(int)
    df.to_csv(self.player_csv, index=False)

def _update_game_stats_csv(self):
    if not PANDAS_AVAILABLE:
        return
    realm_stats = {}
    for profile in self.profiles.values():
        for realm, score in profile.get('realm_scores', {}).items():
            if realm not in realm_stats:
                realm_stats[realm] = {'scores': [], 'players': set(), 'attempts': []}
            realm_stats[realm]['scores'].append(score)
            realm_stats[realm]['players'].add(profile.get('name'))
            attempts = profile.get('realm_play_counts', {}).get(realm, 1)
            realm_stats[realm]['attempts'].append(attempts)
    stats_rows = []
    for realm, data in realm_stats.items():
        if data['scores']:
```

```
            avg_score = sum(data['scores']) / len(data['scores'])
            max_possible_score = 100
            relative_difficulty = 1 - (avg_score / max_possible_score)
            avg_attempts = sum(data['attempts']) / len(data['attempts']) if data['attempts'] else 1
            stats_rows.append({
                'realm_name': realm,
                'average_score': round(avg_score, 2),
                'attempts_before_full_score': round(avg_attempts, 2),
                'relative_difficulty': round(relative_difficulty, 3),
                'number_of_players_who_played_realm': len(data['players'])
            })
    if stats_rows:
        df = pd.DataFrame(stats_rows)
        df.to_csv(self.game_stats_csv, index=False)

def create_profile(self, name: str):
    if name in self.profiles:
        return False, "Profile already exists!"
    self.profiles[name] = {
        "name": name,
        "score": 0,
        "completed_realms": [],
        "realm_scores": {},
        "created_date": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "last_played": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "total_playtime": 0,
        "realms_played": [],
        "realm_play_counts": {},
        "realm_difficulty_stats": {},
        "last_replay_difference": 0,
        "favorite_realm_level": "N/A",
        "last_logout": "N/A"
    }
    self.save_profiles()
    self._update_player_csv(name)
    return True, "Profile created successfully!"

def update_profile(self, name: str, score: int, completed_realms: list, realm_scores: dict) -> bool:
    if name not in self.profiles:
        return False
    self.profiles[name].update({
        "score": score,
        "completed_realms": completed_realms,
        "realm_scores": realm_scores,
```

```
            "last_played": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        })
        self.save_profiles()
        self._update_player_csv(name)
        self._update_game_stats_csv()
        return True

    def delete_profile(self, name: str) -> bool:
        if name in self.profiles:
            del self.profiles[name]
            self.save_profiles()
            if PANDAS_AVAILABLE:
                try:
                    df = pd.read_csv(self.player_csv)
                    df = df[df['username'] != name]
                    df.to_csv(self.player_csv, index=False)
                except Exception:
                    pass
            self._update_game_stats_csv()
            return True
        return False

    def record_realm_play(self, username: str, realm: str, difficulty: str, new_score: int, old_score: int =
None, attempts: int = None):
        if username not in self.profiles:
            return
        profile = self.profiles[username]
        if realm not in profile['realms_played']:
            profile['realms_played'].append(realm)
        if attempts is not None:
            profile['realm_play_counts'][realm] = attempts
        else:
            profile['realm_play_counts'][realm] = profile.get('realm_play_counts', {}).get(realm, 0) + 1
        profile.setdefault('realm_difficulty_stats', {})
        profile['realm_difficulty_stats'].setdefault(realm, {})
        profile['realm_difficulty_stats'][realm].setdefault(difficulty, {'scores': [], 'attempts': 0})
        profile['realm_difficulty_stats'][realm][difficulty]['scores'].append(new_score)
        if attempts is not None:
            profile['realm_difficulty_stats'][realm][difficulty]['attempts'] = attempts
        else:
            profile['realm_difficulty_stats'][realm][difficulty]['attempts'] += 1
        profile['last_replay_difference'] = (new_score - old_score) if old_score is not None else 0
        self._update_favorite_realm_level(username)
        self.save_profiles()
```

```
    self._update_player_csv(username)
    self._update_game_stats_csv()

def _update_favorite_realm_level(self, username: str):
    profile = self.profiles[username]
    stats = profile.get('realm_difficulty_stats', {})
    best_avg = -1
    best_combo = "N/A"
    for realm, difficulties in stats.items():
        for difficulty, data in difficulties.items():
            scores = data.get('scores', [])
            if scores:
                avg_score = sum(scores) / len(scores)
                if avg_score > best_avg:
                    best_avg = avg_score
                    best_combo = f"{realm}:{difficulty} (Avg: {avg_score:.1f})"
    profile['favorite_realm_level'] = best_combo

def get_player_statistics(self, username: str) -> dict:
    if username not in self.profiles:
        return {}
    profile = self.profiles[username]
    return {
        'username': username,
        'total_score': profile.get('score', 0),
        'realms_completed': len(profile.get('completed_realms', [])),
        'realms_played': profile.get('realms_played', []),
        'realm_play_counts': profile.get('realm_play_counts', {}),
        'favorite_realm_level': profile.get('favorite_realm_level', 'N/A'),
        'last_replay_difference': profile.get('last_replay_difference', 0),
        'realm_difficulty_stats': profile.get('realm_difficulty_stats', {}),
        'created_date': profile.get('created_date', ''),
        'last_played': profile.get('last_played', '')
    }

def get_game_statistics(self):
    if not PANDAS_AVAILABLE or not os.path.exists(self.game_stats_csv):
        return pd.DataFrame() if PANDAS_AVAILABLE else None
    try:
        return pd.read_csv(self.game_stats_csv)
    except Exception:
        return pd.DataFrame() if PANDAS_AVAILABLE else None

def get_all_player_data(self):
```

```
    if not PANDAS_AVAILABLE or not os.path.exists(self.player_csv):
        return pd.DataFrame() if PANDAS_AVAILABLE else None
    try:
        df = pd.read_csv(self.player_csv)
        for col in ['total_score', 'realms_completed', 'last_replay_difference']:
            if col in df.columns:
                df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0).astype(int)
        return df
    except Exception:
        return pd.DataFrame() if PANDAS_AVAILABLE else None

def export_analytics_report(self, output_file: str = "analytics_report.txt"):
    if not PANDAS_AVAILABLE:
        messagebox.showerror("Error", "Pandas is required for exporting analytics.")
        return
    try:
        output_path = os.path.join(self.base_dir, output_file)
        player_df = self.get_all_player_data()
        game_df = self.get_game_statistics()
        with open(output_path, 'w', encoding='utf-8') as f:
            f.write("="*70 + "\n")
            f.write("WONDERLAND PROGRAMMING REALMS - ANALYTICS REPORT\n")
            f.write("="*70 + "\n\n")
            f.write("📊 PLAYER STATISTICS\n")
            f.write("-"*70 + "\n")
            if player_df is not None and not player_df.empty:
                f.write(player_df.to_string(index=False))
            else:
                f.write("No player data available.\n")
            f.write("\n\n")
            f.write("🎮 GAME STATISTICS\n")
            f.write("-"*70 + "\n")
            if game_df is not None and not game_df.empty:
                f.write(game_df.to_string(index=False))
            else:
                f.write("No game statistics available.\n")
            f.write("\n\n")
            f.write("🏆 TOP PERFORMERS\n")
            f.write("-"*70 + "\n")
            if player_df is not None and not player_df.empty:
                top_players = player_df.nlargest(5, 'total_score')
                f.write(top_players[['username', 'total_score', 'realms_completed']].to_string(index=False))
            else:
                f.write("No top performer data.\n")
```

```
        messagebox.showinfo("Success", f"Analytics report exported to {output_path}")
        print(f"✅ Analytics report exported to {output_path}")
    except Exception as e:
        messagebox.showerror("Error", f"Failed to export analytics: {e}")
        print(f"Error exporting analytics report: {e}")

class ProfileSelectScreen:
    def __init__(self, root, profile_manager, on_profile_selected):
        self.root = root
        self.root.title("Wonderland Profiles")
        self.profile_manager = profile_manager
        self.on_profile_selected = on_profile_selected
        self.bg_color = '#f0f4f8'
        self.card_color = '#ffffff'
        self.primary_color = '#667eea'
        self.secondary_color = '#764ba2'
        self.accent_color = '#feca57'
        self.text_dark = '#2d3436'
        self.text_light = '#636e72'
        self.root.configure(bg=self.bg_color)
        self.main_frame = tk.Frame(root, bg=self.bg_color)
        self.main_frame.pack(fill='both', expand=True, padx=30, pady=30)
        header_frame = tk.Frame(self.main_frame, bg=self.bg_color)
        header_frame.pack(fill='x', pad=(0, 20))
        title_label = tk.Label(
            header_frame,
            text="🌟 Welcome to Wonderland 🌟",
            font=('Segoe UI', 28, 'bold'),
            fg=self.primary_color,
            bg=self.bg_color
        )
        title_label.pack(pady=(0, 5))
        subtitle_label = tk.Label(
            header_frame,
            text="Select your adventurer profile to begin your journey",
            font=('Segoe UI', 12),
            fg=self.text_light,
            bg=self.bg_color
        )
        subtitle_label.pack()
        self.info_label = tk.Label(
            self.main_frame,
            text="Select a profile to view details...",
            font=('Segoe UI', 10, 'italic'),
```

```
    fg=self.text_light,
    bg=self.bg_color
)
self.info_label.pack(pady=(0, 15))
list_container = tk.Frame(self.main_frame, bg=self.card_color, relief='flat', bd=0)
list_container.pack(pady=10, padx=10, fill='both', expand=True)
border_frame = tk.Frame(list_container, bg='#e1e8ed', bd=0)
border_frame.pack(fill='both', expand=True, padx=2, pady=2)
inner_frame = tk.Frame(border_frame, bg=self.card_color)
inner_frame.pack(fill='both', expand=True, padx=1, pady=1)
scrollbar = tk.Scrollbar(inner_frame, orient=tk.VERTICAL)
self.profiles_listbox = tk.Listbox(
    inner_frame,
    height=10,
    width=50,
    yscrollcommand=scrollbar.set,
    font=('Segoe UI', 12),
    bg=self.card_color,
    fg=self.text_dark,
    selectbackground=self.primary_color,
    selectforeground='white',
    bd=0,
    highlightthickness=0,
    relief='flat',
    activestyle='none'
)
scrollbar.config(command=self.profiles_listbox.yview)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y, padx=(0, 5))
self.profiles_listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=10, pady=10)
self.profiles_listbox.bind('<<ListboxSelect>>', self.show_profile_details)
self.refresh_profiles_list()
btn_frame = tk.Frame(self.main_frame, bg=self.bg_color)
btn_frame.pack(pady=20)
start_btn = tk.Button(
    btn_frame,
    text="🚀 Start Adventure",
    command=self.select_profile
)
style_button(start_btn, bg_color='#10ac84', hover_color='#01a674')
start_btn.pack(side=tk.LEFT, padx=8)
create_btn = tk.Button(
    btn_frame,
    text="➕ Create New Profile",
    command=self.create_new_profile
```

```
        )
        style_button(create_btn, bg_color=self.primary_color, hover_color=self.secondary_color)
        create_btn.pack(side=tk.LEFT, padx=8)
        delete_btn = tk.Button(
            btn_frame,
            text="🗑️ Delete Profile",
            command=self.delete_profile
        )
        style_button(delete_btn, bg_color='#ee5a6f', hover_color='#c23b4d')
        delete_btn.pack(side=tk.LEFT, padx=8)
        self.center_window(root)

    def center_window(self, dialog):
        dialog.update_idletasks()
        x = (dialog.winfo_screenwidth() // 2) - (dialog.winfo_width() // 2)
        y = (dialog.winfo_screenheight() // 2) - (dialog.winfo_height() // 2)
        dialog.geometry(f'+{x}+{y}')

    def refresh_profiles_list(self):
        self.profiles_listbox.delete(0, tk.END)
        if not self.profile_manager.profiles:
            self.profiles_listbox.insert(tk.END, "No profiles yet - Create one to start!")
            self.info_label.config(text="Select a profile to view details...")
        else:
            for name in self.profile_manager.profiles.keys():
                self.profiles_listbox.insert(tk.END, name)

    def show_profile_details(self, event):
        selection = self.profiles_listbox.curselection()
        if not selection:
            self.info_label.config(text="Select a profile to view details...")
            return
        profile_name = self.profiles_listbox.get(selection[0])
        if profile_name in self.profile_manager.profiles:
            profile = self.profile_manager.profiles[profile_name]
            details = (
                f"⭐ Score: {profile['score']} | "
                f"🏰 Realms: {len(profile['completed_realms'])}/{len(Alice4App.get_realm_keys())} | "
                f"🕒 Last Played: {datetime.strptime(profile['last_played'], '%Y-%m-%d
%H:%M:%S').strftime('%b %d, %Y')}"
            )
            self.info_label.config(text=details)

    def create_new_profile(self):
```

```
def save_name(dialog):
    name = name_entry.get().strip()
    if not name:
        messagebox.showerror("Error", "Name cannot be empty!")
        return
    success, message = self.profile_manager.create_profile(name)
    if success:
        dialog.destroy()
        self.refresh_profiles_list()
        self.profiles_listbox.selection_clear(0, tk.END)
        try:
            index = list(self.profile_manager.profiles.keys()).index(name)
            self.profiles_listbox.selection_set(index)
            self.profiles_listbox.see(index)
            self.show_profile_details(None)
        except ValueError:
            pass
        messagebox.showinfo("Success", message)
    else:
        messagebox.showerror("Error", message)
dialog = tk.Toplevel(self.root)
dialog.title("Create New Profile")
dialog.configure(bg=self.bg_color)
dialog.geometry("400x220")
self.center_window(dialog)
dialog.transient(self.root)
dialog.grab_set()
content_frame = tk.Frame(dialog, bg=self.bg_color)
content_frame.pack(fill='both', expand=True, padx=30, pady=30)
tk.Label(
    content_frame,
    text="Enter Adventurer Name:",
    font=('Segoe UI', 13, 'bold'),
    fg=self.text_dark,
    bg=self.bg_color
).pack(pady=(0, 15))
name_entry = tk.Entry(
    content_frame,
    font=('Segoe UI', 12),
    relief='flat',
    bd=0,
    highlightthickness=2,
    highlightbackground='#dfe6e9',
    highlightcolor=self.primary_color
```

```
        )
        name_entry.pack(pady=5, padx=10, ipady=8, fill='x')
        name_entry.focus_set()
        create_btn = tk.Button(
            content_frame,
            text="✨ Create Profile",
            command=lambda: save_name(dialog)
        )
        style_button(create_btn, bg_color=self.primary_color, hover_color=self.secondary_color)
        create_btn.pack(pady=20)
        dialog.bind('<Return>', lambda e: save_name(dialog))

    def select_profile(self):
        selection = self.profiles_listbox.curselection()
        if not selection:
            messagebox.showwarning("No Selection", "Please select or create a profile!")
            return
        profile_name = self.profiles_listbox.get(selection[0])
        if profile_name == "No profiles yet - Create one to start!":
            self.create_new_profile()
            return
        self.main_frame.destroy()
        self.on_profile_selected(profile_name)

    def delete_profile(self):
        selection = self.profiles_listbox.curselection()
        if not selection:
            messagebox.showwarning("No Selection", "Please select a profile to delete!")
            return
        profile_name = self.profiles_listbox.get(selection[0])
        if profile_name == "No profiles yet - Create one to start!":
            return
        if messagebox.askyesno("Confirm Delete", f"Are you sure you want to delete profile
'{profile_name}'?\nThis cannot be undone!"):
            if self.profile_manager.delete_profile(profile_name):
                messagebox.showinfo("Deleted", f"Profile '{profile_name}' has been deleted.")
                self.refresh_profiles_list()
                self.info_label.config(text="Select a profile to view details...")

class Alice4App:
    @staticmethod
    def get_realm_keys():
        return ["python", "cpp", "web", "data"]
```

```
    def __init__(self, root, player_name, profile_manager):
        self.root = root
        self.player_name = player_name
        self.profile_manager = profile_manager
        self.profile_data = self.profile_manager.profiles.get(player_name, {})
        self.session_start = datetime.now()
        self.bg_color = '#f0f4f8'
        self.card_color = '#ffffff'
        self.primary_color = '#667eea'
        self.secondary_color = '#764ba2'
        self.text_dark = '#2d3436'
        self.score = self.profile_data.get('score', 0)
        self.completed_realms = self.profile_data.get('completed_realms', [])
        self.realm_scores = self.profile_data.get('realm_scores', {})
        self.current_realm = None
        self.current_difficulty = "medium"
        self.current_challenge_index = 0
        self.realm_score = 0
        self.current_session_questions = []
        self.realms = {
            "python": {
                "name": "🐍 Python Rabbit Hole",
                "color": "#3498DB",
                "bg_color": "#e3f2fd",
                "accent": "#2196F3",
                "completed": "python" in self.completed_realms,
                "description": "The foundational magic of Python!\nLearn spells for variables, loops, and
magical functions.",
                "character": "🐇",
                "quote": "'Oh dear! Oh dear! I shall be late for Python lessons!'"
            },
            "cpp": {
                "name": "👑 C++ Looking Glass",
                "color": "#4ECDC4",
                "bg_color": "#e0f7fa",
                "accent": "#00BCD4",
                "completed": "cpp" in self.completed_realms,
                "description": "Step through the mirror into the realm of C++!\nMaster memory magic and
object-oriented sorcery.",
                "character": "👑",
                "quote": "'Off with their heads! Unless they know their pointers!'"
            },
            "web": {
                "name": "🎩 Mad Tea Party",
```

```
            "color": "#FFD166",
            "bg_color": "#fff9e6",
            "accent": "#FFC107",
            "completed": "web" in self.completed_realms,
            "description": "Join the Mad Hatter for a chaotic web development tea party!\nCreate magical
interfaces with HTML, CSS & JavaScript.",
            "character": "🎩",
            "quote": "'Why is a raven like a writing desk? Time for CSS!'"
        },
        "data": {
            "name": "📊 Queen's Croquet Ground",
            "color": "#06D6A0",
            "bg_color": "#e8f5e9",
            "accent": "#4CAF50",
            "completed": "data" in self.completed_realms,
            "description": "Play croquet with data at the Queen's court!\nVisualize wonders with NumPy,
Pandas & Matplotlib.",
            "character": "♥",
            "quote": "'All shall visualize data perfectly, or OFF WITH THEIR HEADS!'"
        }
    }
    self.challenges = self.load_challenges()
    self.setup_ui()
    self.root.protocol("WM_DELETE_WINDOW", self.on_closing)

  def load_challenges(self):
    return self.load_questions_from_json()

  def load_questions_from_json(self):
    json_file = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wonderland_questions.json")
    fallback_questions = {
        "python": {
            "easy": [
                {"question": "What keyword is used to define a function in Python?", "options": ["def",
"func", "define", "method"], "correct_answer": 0},
                {"question": "Which data type is ordered, changeable, and allows duplicate members?",
"options": ["set", "tuple", "list", "dictionary"], "correct_answer": 2}
            ],
            "medium": [
                {"question": "What is the result of 'hello' * 3?", "options": ["hellohellohello", "Error",
"3hello", "hello 3"], "correct_answer": 0}
            ],
            "hard": []
        },
```

```
            "cpp": {"easy": [], "medium": [], "hard": []},
            "web": {"easy": [], "medium": [], "hard": []},
            "data": {"easy": [], "medium": [], "hard": []}
        }
        try:
            with open(json_file, "r", encoding="utf-8") as f:
                questions_data = json.load(f)
                print(f"✅ Successfully loaded questions from {json_file}")
                return questions_data
        except FileNotFoundError:
            print(f"⚠️ Warning: {json_file} not found. Using fallback question set.")
            messagebox.showwarning("File Missing", f"The file 'wonderland_questions.json' was not found.
Using a limited internal question set. Challenges will be minimal.")
            return fallback_questions
        except json.JSONDecodeError as e:
            print(f"❌ Error parsing {json_file}: {e}")
            messagebox.showerror("JSON Error", f"Error reading questions file: {e}\nUsing fallback set.")
            return fallback_questions
        except Exception as e:
            print(f"❌ Unexpected error loading questions: {e}")
            messagebox.showerror("Error", f"Could not load questions: {e}\nUsing fallback set.")
            return fallback_questions

    def save_profile(self):
        self.profile_manager.update_profile(self.player_name, self.score, self.completed_realms,
self.realm_scores)

    def on_closing(self):
        try:
            session_end = datetime.now()
            session_start = getattr(self, 'session_start', session_end)
            duration = session_end - session_start
            seconds = int(duration.total_seconds())
            profile = self.profile_manager.profiles.get(self.player_name)
            if profile is not None:
                profile['total_playtime'] = profile.get('total_playtime', 0) + seconds
                profile['last_logout'] = session_end.strftime("%Y-%m-%d %H:%M:%S")
                self.profile_manager.save_profiles()
                try:
                    self.profile_manager._update_player_csv(self.player_name)
                    self.profile_manager._update_game_stats_csv()
                except Exception:
                    pass
        except Exception as e:
```

```
            print(f"Error while recording session end: {e}")
        self.save_profile()
        self.root.destroy()

    def logout(self):
        try:
            session_end = datetime.now()
            session_start = getattr(self, 'session_start', session_end)
            duration = session_end - session_start
            seconds = int(duration.total_seconds())
            profile = self.profile_manager.profiles.get(self.player_name)
            if profile is not None:
                profile['total_playtime'] = profile.get('total_playtime', 0) + seconds
                profile['last_logout'] = session_end.strftime("%Y-%m-%d %H:%M:%S")
                self.profile_manager.save_profiles()
                try:
                    self.profile_manager._update_player_csv(self.player_name)
                    self.profile_manager._update_game_stats_csv()
                except Exception:
                    pass
        except Exception as e:
            print(f"Error while logging out: {e}")
        self.save_profile()
        for widget in self.root.winfo_children():
            widget.destroy()
        ProfileSelectScreen(self.root, self.profile_manager, lambda name: Alice4App(self.root, name,
self.profile_manager))

    def center_window(self, dialog=None):
        target = dialog if dialog else self.root
        target.update_idletasks()
        x = (target.winfo_screenwidth() // 2) - (target.winfo_width() // 2)
        y = (target.winfo_screenheight() // 2) - (target.winfo_height() // 2)
        target.geometry(f'+{x}+{y}')

    def open_settings(self):
        profile = self.profile_manager.profiles.get(self.player_name, {})
        settings_win = tk.Toplevel(self.root)
        settings_win.title("⚙️ Settings & Session")
        settings_win.geometry("500x450")
        settings_win.configure(bg=self.bg_color)
        settings_win.transient(self.root)
        header_frame = tk.Frame(settings_win, bg=self.primary_color)
        header_frame.pack(fill='x')
```

```
    tk.Label(header_frame, text=f"⚙️ Settings for {self.player_name}", font=('Segoe UI', 16, 'bold'),
fg='white', bg=self.primary_color, pady=15).pack()
    content_frame = tk.Frame(settings_win, bg=self.bg_color)
    content_frame.pack(fill='both', expand=True, padx=30, pady=20)
    session_start = getattr(self, 'session_start', None)
    start_str = session_start.strftime("%Y-%m-%d %H:%M:%S") if session_start else 'N/A'
    last_logout = profile.get('last_logout', 'N/A')
    total_playtime = profile.get('total_playtime', 0)
    info_items = [("🕒 Logged in at:", start_str), ("🕒 Last logout:", last_logout)]
    for label, value in info_items:
        item_frame = tk.Frame(content_frame, bg=self.card_color, relief='flat')
        item_frame.pack(fill='x', pady=5)
        tk.Label(item_frame, text=label, font=('Segoe UI', 11, 'bold'), fg=self.text_dark,
bg=self.card_color, anchor='w').pack(side='left', padx=15, pady=10)
        tk.Label(item_frame, text=value, font=('Segoe UI', 11), fg='#636e72', bg=self.card_color,
anchor='e').pack(side='right', padx=15, pady=10)
    try:
        hours, remainder = divmod(int(total_playtime), 3600)
        minutes, seconds = divmod(remainder, 60)
        tp_str = f"{hours:02d}:{minutes:02d}:{seconds:02d}"
    except Exception:
        tp_str = str(total_playtime)
    playtime_frame = tk.Frame(content_frame, bg=self.card_color, relief='flat')
    playtime_frame.pack(fill='x', pady=5)
    tk.Label(playtime_frame, text="⏱️ Total playtime:", font=('Segoe UI', 11, 'bold'), fg=self.text_dark,
bg=self.card_color, anchor='w').pack(side='left', padx=15, pady=10)
    tk.Label(playtime_frame, text=f"{tp_str} (HH:MM:SS)", font=('Segoe UI', 11), fg='#636e72',
bg=self.card_color, anchor='e').pack(side='right', padx=15, pady=10)
    tk.Label(content_frame, text="📊 Quick Stats", font=('Segoe UI', 13, 'bold'), fg=self.primary_color,
bg=self.bg_color).pack(pady=(15, 10))
    stats = self.profile_manager.get_player_statistics(self.player_name)
    stats_items = [("Total Score:", stats.get('total_score', 0)), ("Realms Completed:",
stats.get('realms_completed', 0)), ("Favorite Level:", stats.get('favorite_realm_level', 'N/A'))]
    for label, value in stats_items:
        stat_frame = tk.Frame(content_frame, bg=self.card_color)
        stat_frame.pack(fill='x', pady=3)
        tk.Label(stat_frame, text=label, font=('Segoe UI', 10), fg=self.text_dark, bg=self.card_color,
anchor='w').pack(side='left', padx=15, pady=8)
        tk.Label(stat_frame, text=str(value), font=('Segoe UI', 10, 'bold'), fg=self.primary_color,
bg=self.card_color, anchor='e').pack(side='right', padx=15, pady=8)
    close_btn = tk.Button(content_frame, text="Close", command=settings_win.destroy)
    style_button(close_btn, bg_color=self.primary_color, hover_color=self.secondary_color)
    close_btn.pack(pady=20)
    self.center_window(settings_win)
```

```
    def setup_ui(self):
        self.root.title("Wonderland Programming Realms")
        self.root.geometry("1100x850")
        self.root.configure(bg=self.bg_color)
        if hasattr(self, 'main_frame'):
            self.main_frame.destroy()
        self.main_frame = tk.Frame(self.root, bg=self.bg_color)
        self.main_frame.pack(fill='both', expand=True)
        header_frame = tk.Frame(self.main_frame, bg='#667eea', height=120)
        header_frame.pack(fill='x', side='top')
        header_frame.pack_propagate(False)
        title_container = tk.Frame(header_frame, bg='#667eea')
        title_container.pack(expand=True)
        title_label = tk.Label(title_container, text="🌟 Wonderland Programming Realms 🌟", font=('Segoe
UI', 32, 'bold'), fg='white', bg='#667eea')
        title_label.pack(pady=(10, 0))
        subtitle_label = tk.Label(title_container, text="Where Code Meets Magic ✨", font=('Segoe UI', 14,
'italic'), fg='#ffeaa7', bg='#667eea')
        subtitle_label.pack(pady=(0, 5))
        settings_btn = tk.Button(header_frame, text="⚙️", font=('Segoe UI', 16),
command=self.open_settings, relief='flat', bg='#764ba2', fg='white', cursor='hand2', width=3, height=1)
        settings_btn.place(relx=0.95, rely=0.5, anchor='e')
        def settings_hover_in(e):
            settings_btn.config(bg='#5e3b8a')
        def settings_hover_out(e):
            settings_btn.config(bg='#764ba2')
        settings_btn.bind("<Enter>", settings_hover_in)
        settings_btn.bind("<Leave>", settings_hover_out)
        logout_btn = tk.Button(header_frame, text="🔓", font=('Segoe UI', 16), command=self.logout,
relief='flat', bg='#e17055', fg='white', cursor='hand2', width=3, height=1)
        logout_btn.place(relx=0.88, rely=0.5, anchor='e')
        def logout_hover_in(e):
            logout_btn.config(bg='#d63031')
        def logout_hover_out(e):
            logout_btn.config(bg='#e17055')
        logout_btn.bind("<Enter>", logout_hover_in)
        logout_btn.bind("<Leave>", logout_hover_out)
        content_frame = tk.Frame(self.main_frame, bg=self.bg_color)
        content_frame.pack(fill='both', expand=True, padx=30, pady=20)
        info_card = tk.Frame(content_frame, bg=self.card_color, relief='flat')
        info_card.pack(fill='x', pady=(0, 20))
        shadow_frame = tk.Frame(content_frame, bg='#dfe6e9')
        shadow_frame.place(in_=info_card, relx=0.01, rely=0.01, relwidth=1, relheight=1)
```

```
        info_card.lift()
        info_inner = tk.Frame(info_card, bg=self.card_color)
        info_inner.pack(fill='x', padx=20, pady=15)
        self.name_label = tk.Label(info_inner, text=f"✨ Adventurer: {self.player_name}", font=('Segoe
UI', 14, 'bold'), fg=self.primary_color, bg=self.card_color)
        self.name_label.pack(side='left', padx=10)
        self.score_label = tk.Label(info_inner, text=f"🏆 Magic Score: {self.score}", font=('Segoe UI', 14,
'bold'), fg='#e17055', bg=self.card_color)
        self.score_label.pack(side='right', padx=10)
        self.progress_label = tk.Label(info_inner, text="", font=('Segoe UI', 13), fg='#00b894',
bg=self.card_color)
        self.progress_label.pack(side='right', padx=20)
        self.update_progress()
        action_frame = tk.Frame(content_frame, bg=self.bg_color)
        action_frame.pack(fill='x', pady=(0, 20))
        buttons_config = [
            ("👤 Profile Dashboard", self.show_profile_dashboard, '#6c5ce7'),
            ("🧪 Python Spellbook", self.show_python_spellbook, '#a29bfe'),
            ("📈 Data Visualization", self.show_visualization_demo, '#00b894'),
            ("💾 Export Analytics", self.profile_manager.export_analytics_report, '#fd79a8')
        ]
        for text, command, color in buttons_config:
            btn = tk.Button(action_frame, text=text, command=command)
            hover = self.darken_color(color)
            style_button(btn, bg_color=color, hover_color=hover)
            btn.pack(side='left', padx=5)
        realms_header = tk.Label(content_frame, text="🗺️ Explore the Realms", font=('Segoe UI', 20,
'bold'), fg=self.text_dark, bg=self.bg_color)
        realms_header.pack(pady=(10, 15))
        realms_container = tk.Frame(content_frame, bg=self.bg_color)
        realms_container.pack(pady=10, fill='both', expand=True)
        realms_container.grid_columnconfigure(0, weight=1)
        realms_container.grid_columnconfigure(1, weight=1)
        realms_container.grid_columnconfigure(2, weight=1)
        realms_container.grid_columnconfigure(3, weight=1)
        self.realm_buttons = {}
        col = 0
        for realm_key, realm in self.realms.items():
            realm_card = self.create_realm_card(realms_container, realm_key, realm)
            realm_card.grid(row=0, column=col, padx=10, pady=10, sticky='nsew')
            self.realm_buttons[realm_key] = realm_card
            col += 1
        self.center_window()
```

```
    def darken_color(self, hex_color, factor=0.8):
        hex_color = hex_color.lstrip('#')
        r, g, b = tuple(int(hex_color[i:i+2], 16) for i in (0, 2, 4))
        r = int(r * factor)
        g = int(g * factor)
        b = int(b * factor)
        return f'#{r:02x}{g:02x}{b:02x}'

    def create_realm_card(self, parent, realm_key, realm):
        if realm['completed']:
            bg_color = '#d4edda'
            border_color = '#28a745'
            status_text = "✅ CONQUERED"
            status_bg = '#28a745'
        else:
            bg_color = realm['bg_color']
            border_color = realm['accent']
            status_text = "🔒 CHALLENGE"
            status_bg = '#e74c3c'
        card_frame = tk.Frame(parent, bg=border_color, relief='flat', width=240, height=300,
cursor='hand2')
        card_frame.pack_propagate(False)
        inner_frame = tk.Frame(card_frame, bg=bg_color)
        inner_frame.pack(fill='both', expand=True, padx=3, pady=3)
        status_label = tk.Label(inner_frame, text=status_text, font=('Segoe UI', 9, 'bold'), fg='white',
bg=status_bg, pady=5)
        status_label.pack(fill='x', pady=(5, 10))
        char_label = tk.Label(inner_frame, text=realm['character'], font=('Segoe UI', 48), bg=bg_color)
        char_label.pack(pady=5)
        name_label = tk.Label(inner_frame, text=realm['name'], font=('Segoe UI', 13, 'bold'),
fg=realm['color'], bg=bg_color, wraplength=200, justify='center')
        name_label.pack(pady=5)
        quote_label = tk.Label(inner_frame, text=realm['quote'], font=('Segoe UI', 9, 'italic'), fg='#636e72',
bg=bg_color, wraplength=200, justify='center')
        quote_label.pack(pady=5, padx=10)
        if realm_key in self.realm_scores:
            score_label = tk.Label(inner_frame, text=f"Best: {self.realm_scores[realm_key]}/100",
font=('Segoe UI', 10, 'bold'), fg=realm['color'], bg=bg_color)
            score_label.pack(pady=5)
        def on_click(event):
            self.enter_realm(realm_key)
        def on_enter(event):
            card_frame.config(bg=self.darken_color(border_color, 0.7))
        def on_leave(event):
```

```
        card_frame.config(bg=border_color)
    for widget in [card_frame, inner_frame, status_label, char_label, name_label, quote_label]:
        widget.bind('<Button-1>', on_click)
        widget.bind('<Enter>', on_enter)
        widget.bind('<Leave>', on_leave)
    return card_frame

def recalculate_total_score(self):
    new_total_score = sum(self.realm_scores.values())
    self.score = new_total_score
    self.score_label.config(text=f"🏆 Magic Score: {self.score}")
    self.update_progress()
    self.save_profile()

def reset_realm_score_explicitly(self, window, realm_key):
    realm_name = self.realms[realm_key]['name']
    if messagebox.askyesno("Confirm Score Reset", f"Are you sure you want to PERMANENTLY reset
the score for {realm_name} to 0?\nThis action cannot be undone!", parent=window):
        if realm_key in self.realm_scores and self.realm_scores[realm_key] != 0:
            self.realm_scores[realm_key] = 0
            if realm_key in self.completed_realms:
                self.completed_realms.remove(realm_key)
                self.realms[realm_key]['completed'] = False
            self.recalculate_total_score()
            messagebox.showinfo("Reset Complete", f"Score for {realm_name} has been reset to 0.",
parent=window)
            self.setup_ui()
        else:
            messagebox.showinfo("Info", "Realm score is already 0 or not set.", parent=window)
        window.destroy()

def _start_realm_quiz_flow(self, parent_window, realm_key):
    parent_window.destroy()
    self.current_realm = realm_key
    self.realm_score = 0
    self.show_difficulty_selector(realm_key)

def enter_realm(self, realm_key):
    realm_data = self.realms[realm_key]
    realm_window = tk.Toplevel(self.root)
    realm_window.title(f"{realm_data['name']}")
    realm_window.geometry("600x500")
    realm_window.configure(bg=self.bg_color)
    realm_window.transient(self.root)
```

```
        realm_window.grab_set()
        header = tk.Frame(realm_window, bg=realm_data['accent'], height=80)
        header.pack(fill='x')
        header.pack_propagate(False)
        tk.Label(header, text=realm_data['name'], font=('Segoe UI', 22, 'bold'), fg='white',
bg=realm_data['accent']).pack(expand=True)
        content = tk.Frame(realm_window, bg=self.bg_color)
        content.pack(fill='both', expand=True, padx=30, pady=20)
        tk.Label(content, text=realm_data['description'], font=('Segoe UI', 11), fg=self.text_dark,
bg=self.bg_color, wraplength=500, justify='center').pack(pady=15)
        current_score = self.realm_scores.get(realm_key)
        score_text = f"🏆 Current Best Score: {current_score} / 100" if current_score is not None else "🆕
No score recorded yet"
        start_btn_text = "🔄 Replay Realm (Overwrite Score)" if current_score is not None else "🚀 Start
New Challenge"
        start_btn_color = '#e17055' if current_score is not None else '#00b894'
        start_hover = self.darken_color(start_btn_color)
        tk.Label(content, text=score_text, font=('Segoe UI', 13, 'bold'), fg=realm_data['color'],
bg=self.bg_color).pack(pady=10)
        start_btn = tk.Button(content, text=start_btn_text, command=lambda:
self._start_realm_quiz_flow(realm_window, realm_key))
        style_button(start_btn, bg_color=start_btn_color, hover_color=start_hover)
        start_btn.pack(pady=20)
        if current_score is not None and current_score != 0:
            tk.Label(content, text="or...", font=('Segoe UI', 9), fg='#636e72', bg=self.bg_color).pack(pady=5)
            reset_btn = tk.Button(content, text="⚠️ Reset Realm Score to 0", command=lambda:
self.reset_realm_score_explicitly(realm_window, realm_key))
            style_button(reset_btn, bg_color='#d63031', hover_color='#a02828')
            reset_btn.pack(pady=5)
        back_btn = tk.Button(content, text="← Back to Map", command=lambda: realm_window.destroy())
        style_button(back_btn, bg_color='#636e72', hover_color='#4a4a4a')
        back_btn.pack(pady=15)
        self.center_window(realm_window)

    def show_difficulty_selector(self, realm_key):
        realm_data = self.realms[realm_key]
        diff_window = tk.Toplevel(self.root)
        diff_window.title(f"⚡ Select Difficulty - {realm_data['name']}")
        diff_window.geometry("550x450")
        diff_window.configure(bg=self.bg_color)
        diff_window.transient(self.root)
        diff_window.grab_set()
        header = tk.Frame(diff_window, bg=realm_data['accent'])
        header.pack(fill='x', pady=(0, 20))
```

```
        tk.Label(header, text="⚡ Choose Your Challenge Level ⚡", font=('Segoe UI', 18, 'bold'),
fg='white', bg=realm_data['accent'], pady=20).pack()
        content = tk.Frame(diff_window, bg=self.bg_color)
        content.pack(fill='both', expand=True, padx=40, pady=10)
        tk.Label(content, text="Select the difficulty that matches your skill level!", font=('Segoe UI', 11),
fg=self.text_dark, bg=self.bg_color).pack(pady=10)
        def start_quiz_session(dialog, difficulty):
            dialog.destroy()
            self._start_quiz_session(dialog, realm_key, difficulty)
        difficulties = [
            ("🌱 Easy (Beginner)", 'easy', '#00b894', '#019874'),
            ("⚡ Medium (Adventurer)", 'medium', '#fdcb6e', '#e0b052'),
            ("🔥 Hard (Master)", 'hard', '#d63031', '#a02828')
        ]
        for text, level, color, hover in difficulties:
            btn = tk.Button(content, text=text, command=lambda l=level: start_quiz_session(diff_window, l))
            style_button(btn, bg_color=color, hover_color=hover)
            btn.pack(pady=10, fill='x', ipady=5)
        self.center_window(diff_window)

    def _start_quiz_session(self, parent_window, realm_key, difficulty):
        self.current_realm = realm_key
        self.current_difficulty = difficulty
        self.realm_score = 0
        self.current_challenge_index = 0
        challenges_for_level = self.challenges.get(realm_key, {}).get(difficulty, [])[:]
        random.shuffle(challenges_for_level)
        self.current_session_questions = challenges_for_level[:5]
        if not self.current_session_questions:
            messagebox.showerror("No Challenges", f"No {difficulty.title()} challenges found for the
{self.realms[realm_key]['name']} realm.")
            self.setup_ui()
            return
        self.display_challenge(self.current_session_questions[0])

    def display_challenge(self, challenge):
        realm_key = self.current_realm
        realm_data = self.realms[realm_key]
        self.challenge_window = tk.Toplevel(self.root)
        self.challenge_window.title(f"Challenge {self.current_challenge_index +
1}/{len(self.current_session_questions)}")
        self.challenge_window.geometry("850x650")
        self.challenge_window.configure(bg=self.bg_color)
        self.challenge_window.transient(self.root)
```

```
        self.challenge_window.grab_set()
        header = tk.Frame(self.challenge_window, bg=realm_data['accent'])
        header.pack(fill='x')
        difficulty_colors = {"easy": "#00b894", "medium": "#fdcb6e", "hard": "#d63031"}
        tk.Label(header, text=f"Realm: {realm_data['name']} | Difficulty: {self.current_difficulty.upper()}",
font=('Segoe UI', 14, 'bold'), fg='white', bg=realm_data['accent'], pady=15).pack()
        self.session_score_label = tk.Label(header, text=f"Session Score: {self.realm_score}", font=('Segoe
UI', 12, 'bold'), fg='#ffeaa7', bg=realm_data['accent'], pady=5)
        self.session_score_label.pack()
        content = tk.Frame(self.challenge_window, bg=self.bg_color)
        content.pack(fill='both', expand=True, padx=30, pady=20)
        question_card = tk.Frame(content, bg=self.card_color, relief='flat')
        question_card.pack(fill='x', pady=10)
        tk.Label(question_card, text=f"❓ Question:", font=('Segoe UI', 12, 'bold'), fg=realm_data['color'],
bg=self.card_color, anchor='w').pack(fill='x', padx=20, pady=(15, 5))
        tk.Label(question_card, text=challenge['question'], font=('Segoe UI', 12), fg=self.text_dark,
bg=self.card_color, wraplength=750, justify='left', anchor='w').pack(fill='x', padx=20, pady=(0, 15))
        tk.Label(content, text="Select your answer:", font=('Segoe UI', 11, 'bold'), fg=self.text_dark,
bg=self.bg_color).pack(pady=(10, 5))
        self.selected_option = tk.IntVar(value=-1)
        self.option_frames = []
        def on_option_select():
            for frame, _, _ in self.option_frames:
                frame.config(relief='flat', bg=self.card_color, highlightthickness=0)
            selected_index = self.selected_option.get()
            if selected_index != -1:
                selected_frame, _, _ = self.option_frames[selected_index]
                selected_frame.config(relief='solid', bg=realm_data['bg_color'], highlightthickness=2,
highlightbackground=realm_data['accent'])
        for i, option in enumerate(challenge['options']):
            option_frame = tk.Frame(content, bg=self.card_color, relief='flat', cursor='hand2')
            option_frame.pack(fill='x', pady=5)
            rb = tk.Radiobutton(option_frame, variable=self.selected_option, value=i, text=f"  {chr(65+i)}) 
{option}", command=on_option_select, bg=self.card_color, activebackground=self.card_color,
fg=self.text_dark, selectcolor=realm_data['accent'], font=('Segoe UI', 11), anchor='w', cursor='hand2')
            rb.pack(fill='x', padx=15, pady=10)
            self.option_frames.append((option_frame, rb, rb))
        submit_btn = tk.Button(content, text="✅ Submit Answer & Next", command=lambda:
self.check_answer(challenge.get('correct_answer', challenge.get('answer', 0))))
        style_button(submit_btn, bg_color='#00b894', hover_color='#019874')
        submit_btn.pack(pady=20)
        self.center_window(self.challenge_window)

    def check_answer(self, correct_answer_index):
```

```
        user_selection = self.selected_option.get()
        score_per_question = 100 // len(self.current_session_questions) if self.current_session_questions
else 20
        if user_selection == -1:
            messagebox.showwarning("Select Answer", "Please select an answer before submitting.",
parent=self.challenge_window)
            return
        if user_selection == correct_answer_index:
            self.realm_score += score_per_question
            messagebox.showinfo("Correct!", f"✨ Magic works! Your answer is correct.
(+{score_per_question} Score)", parent=self.challenge_window)
        else:
            messagebox.showerror("Incorrect!", "❌ The spell failed. Try again next time.",
parent=self.challenge_window)
        if hasattr(self, 'session_score_label') and self.session_score_label.winfo_exists():
            self.session_score_label.config(text=f"Session Score: {self.realm_score}")
        self.challenge_window.destroy()
        self.next_question()

    def next_question(self):
        self.current_challenge_index += 1
        if self.current_challenge_index < len(self.current_session_questions):
            self.display_challenge(self.current_session_questions[self.current_challenge_index])
        else:
            self.end_realm_session()

    def end_realm_session(self):
        realm_key = self.current_realm
        final_realm_score = self.realm_score
        old_score = self.realm_scores.get(realm_key, 0)
        self.realm_scores[realm_key] = final_realm_score
        if final_realm_score > 0 and realm_key not in self.completed_realms:
            self.completed_realms.append(realm_key)
            self.realms[realm_key]['completed'] = True
        self.recalculate_total_score()
        try:
            self.profile_manager.record_realm_play(self.player_name, realm_key, self.current_difficulty,
new_score=final_realm_score, old_score=old_score, attempts=self.current_challenge_index)
        except Exception as e:
            print(f"Error recording realm play analytics: {e}")
        current_realm_name = self.realms[realm_key]['name']
        messagebox.showinfo("Realm Conquered!", f"Congratulations! You've completed the
{current_realm_name} realm.\nYour score for this attempt: {final_realm_score}\nYour new Total Magic
Score: {self.score}")
```

```
        self.setup_ui()
        if len(self.completed_realms) == len(self.realms):
            self.show_completion()
        self.current_realm = None
        self.current_challenge_index = 0
        self.realm_score = 0
        self.current_session_questions = []

    def update_progress(self):
        completed_count = len(self.completed_realms)
        total_count = len(self.realms)
        progress_text = f"🏰 Realms Conquered: {completed_count}/{total_count}"
        if total_count > 0:
            percentage = (completed_count / total_count) * 100
            progress_text += f" ({percentage:.0f}%)"
        self.progress_label.config(text=progress_text)

    def show_completion(self):
        messagebox.showinfo("Grand Victory!", f"👑🏆 Congratulations, Adventurer! 🏆👑\nYou have
successfully conquered all Programming Realms in Wonderland!\nYour final Magic Score is:
{self.score}")

    def show_python_spellbook(self):
        spellbook_window = tk.Toplevel(self.root)
        spellbook_window.title("Python Advanced Spellbook")
        spellbook_window.geometry("900x800")
        spellbook_window.configure(bg=self.bg_color)
        spellbook_window.transient(self.root)
        header = tk.Frame(spellbook_window, bg='#8E44AD')
        header.pack(fill='x')
        tk.Label(header, text="🧙 Python Spellbook - Advanced Magic 🧙", font=('Segoe UI', 20, 'bold'),
fg='white', bg='#8E44AD', pady=20).pack()
        notebook = ttk.Notebook(spellbook_window)
        notebook.pack(fill='both', expand=True, padx=20, pady=10)
        recursion_frame = tk.Frame(notebook, bg=self.bg_color)
        notebook.add(recursion_frame, text="🔄 Recursion")
        self.create_recursion_demo(recursion_frame)
        functional_frame = tk.Frame(notebook, bg=self.bg_color)
        notebook.add(functional_frame, text="⚡ Functional")
        self.create_functional_demo(functional_frame)
        regex_frame = tk.Frame(notebook, bg=self.bg_color)
        notebook.add(regex_frame, text="🔍 Regex")
        self.create_regex_demo(regex_frame)
        exception_frame = tk.Frame(notebook, bg=self.bg_color)
```

```
        notebook.add(exception_frame, text="🛡️ Exceptions")
        self.create_exception_demo(exception_frame)
        self.center_window(spellbook_window)

    def create_recursion_demo(self, parent):
        title = tk.Label(parent, text="🔄 Recursion - Functions Calling Themselves (Factorial Example)",
font=('Segoe UI', 14, 'bold'), fg='#8E44AD', bg=self.bg_color)
        title.pack(pady=15)
        result_text = scrolledtext.ScrolledText(parent, height=20, width=100, font=('Courier', 10),
bg='#2d3436', fg='#00FF00', wrap=tk.WORD)
        result_text.pack(pady=15, padx=20, fill='both', expand=True)
        def run_recursion_demo():
            result_text.delete(1.0, tk.END)
            result_text.insert(tk.END, "🔄 RECURSION MAGIC DEMONSTRATION\n")
            result_text.insert(tk.END, "="*80 + "\n\n")
            code = """
def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)

result_5 = factorial_recursive(5)
result_7 = factorial_recursive(7)
"""
            result_text.insert(tk.END, code + "\n")
            try:
                def factorial_recursive(n):
                    if n <= 1:
                        return 1
                    return n * factorial_recursive(n - 1)
                result_5 = factorial_recursive(5)
                result_7 = factorial_recursive(7)
                result_text.insert(tk.END, f"factorial_recursive(5) = {result_5}\n")
                result_text.insert(tk.END, f"factorial_recursive(7) = {result_7}\n")
            except Exception as e:
                result_text.insert(tk.END, f"Error during execution: {e}\n")
            result_text.insert(tk.END, "\n" + "="*80 + "\n")
            result_text.insert(tk.END, "🎯 KEY RECURSION CONCEPTS:\n")
            result_text.insert(tk.END, "="*80 + "\n")
            result_text.insert(tk.END, "✓ Base Case: The condition that stops the recursion.\n")
            result_text.insert(tk.END, "✓ Recursive Step: The part that calls the function itself.\n")
            result_text.insert(tk.END, "✓ Use caution: Infinite recursion leads to stack overflow!\n\n")
        run_btn = tk.Button(parent, text="▶️ Run Recursion Demo", command=run_recursion_demo)
        style_button(run_btn, bg_color='#3498DB', hover_color='#2980b9')
```

```
        run_btn.pack(pady=10)
        run_recursion_demo()

    def create_functional_demo(self, parent):
        title = tk.Label(parent, text="⚡ Functional Magic - Comprehensions, Lambda, Map, Filter,
Reduce", font=('Segoe UI', 14, 'bold'), fg='#8E44AD', bg=self.bg_color)
        title.pack(pady=15)
        result_text = scrolledtext.ScrolledText(parent, height=20, width=100, font=('Courier', 10),
bg='#2d3436', fg='#00FF00', wrap=tk.WORD)
        result_text.pack(pady=15, padx=20, fill='both', expand=True)
        def run_functional_demo():
            result_text.delete(1.0, tk.END)
            result_text.insert(tk.END, "⚡ FUNCTIONAL MAGIC DEMONSTRATION\n")
            result_text.insert(tk.END, "="*80 + "\n\n")
            result_text.insert(tk.END, "🧩 Example 1: List Comprehension\n")
            result_text.insert(tk.END, "-" * 80 + "\n")
            code1 = "numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]\nresult = [x**2 for x in numbers if x % 2 == 0]"
            result_text.insert(tk.END, code1 + "\n")
            numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
            result = [x**2 for x in numbers if x % 2 == 0]
            result_text.insert(tk.END, f"Result: {result}\n")
            result_text.insert(tk.END, "\n🧩 Example 2: Lambda Functions\n")
            result_text.insert(tk.END, "-" * 80 + "\n")
            code2 = "square = lambda x: x * x\nadd = lambda x, y: x + y"
            result_text.insert(tk.END, code2 + "\n")
            square = lambda x: x * x
            add = lambda x, y: x + y
            result_text.insert(tk.END, f"square(5) = {square(5)}\n")
            result_text.insert(tk.END, f"add(10, 20) = {add(10, 20)}\n")
            result_text.insert(tk.END, "\n" + "="*80 + "\n")
            result_text.insert(tk.END, "🗺️ MAP, 🔍 FILTER, 🧱 REDUCE\n")
            result_text.insert(tk.END, "="*80 + "\n\n")
            result_text.insert(tk.END, "Example 3: Map\n")
            result_text.insert(tk.END, "-" * 80 + "\n")
            numbers = [1, 2, 3, 4]
            doubled = list(map(lambda x: x * 2, numbers))
            result_text.insert(tk.END, f"Original: {numbers}\nDoubled: {doubled}\n")
            result_text.insert(tk.END, "\nExample 4: Filter\n")
            result_text.insert(tk.END, "-" * 80 + "\n")
            numbers = [1, 5, 10, 2]
            large_nums = list(filter(lambda x: x > 4, numbers))
            result_text.insert(tk.END, f"Original: {numbers}\nFiltered: {large_nums}\n")
            result_text.insert(tk.END, "\nExample 5: Reduce\n")
            result_text.insert(tk.END, "-" * 80 + "\n")
```

```
        magic_numbers = [10, 20, 30]
        total = reduce(lambda x, y: x + y, magic_numbers)
        result_text.insert(tk.END, f"Sum: {total}\n")
        result_text.insert(tk.END, "\n" + "="*80 + "\n")
        result_text.insert(tk.END, "🎯 KEY FUNCTIONAL CONCEPTS:\n")
        result_text.insert(tk.END, "="*80 + "\n")
        result_text.insert(tk.END, "✓ Comprehensions: Concise syntax for creating lists/sets/dicts.\n")
        result_text.insert(tk.END, "✓ Lambda: One-line, throwaway functions.\n")
        result_text.insert(tk.END, "✓ Map/Filter/Reduce: Functional tools for data transformation\n\n")
    run_btn = tk.Button(parent, text="▶️ Run Functional Demo", command=run_functional_demo)
    style_button(run_btn, bg_color='#3498DB', hover_color='#2980b9')
    run_btn.pack(pady=10)
    run_functional_demo()

  def create_regex_demo(self, parent):
    title = tk.Label(parent, text="🔍 Regex Patterns - Searching the Wonderland Forest", font=('Segoe
UI', 14, 'bold'), fg='#8E44AD', bg=self.bg_color)
    title.pack(pady=15)
    result_text = scrolledtext.ScrolledText(parent, height=20, width=100, font=('Courier', 10),
bg='#2d3436', fg='#00FF00', wrap=tk.WORD)
    result_text.pack(pady=15, padx=20, fill='both', expand=True)
    def run_regex_demo():
        result_text.delete(1.0, tk.END)
        result_text.insert(tk.END, "🔍 REGULAR EXPRESSION MAGIC DEMONSTRATION\n")
        result_text.insert(tk.END, "="*80 + "\n\n")
        result_text.insert(tk.END, "📜 Example 1: Finding Phone Numbers (re.search)\n")
        result_text.insert(tk.END, "-" * 80 + "\n")
        text = "Call the White Rabbit at 555-123-4567 for urgent tea time!"
        phone_pattern = r'\d{3}-\d{3}-\d{4}'
        match = re.search(phone_pattern, text)
        result_text.insert(tk.END, f"Text: {text}\n")
        result_text.insert(tk.END, f"Pattern: {phone_pattern}\n")
        result_text.insert(tk.END, f"Found: {match.group(0) if match else 'Not Found'}\n")
        result_text.insert(tk.END, "\n📜 Example 2: Finding all words starting with 'R'\n")
        result_text.insert(tk.END, "-" * 80 + "\n")
        text = "The Red Queen rode a Royal Unicorn in the Realm."
        pattern = r'R[a-z]+'
        matches = re.findall(pattern, text)
        result_text.insert(tk.END, f"Text: {text}\n")
        result_text.insert(tk.END, f"Pattern: {pattern}\n")
        result_text.insert(tk.END, f"Found Matches: {matches}\n")
        result_text.insert(tk.END, "\n📜 Example 3: Replacing a word (re.sub)\n")
        result_text.insert(tk.END, "-" * 80 + "\n")
        text = "The Jabberwocky is here."
```

```
            new_text = re.sub(r'Jabberwocky', 'Bandersnatch', text)
            result_text.insert(tk.END, f"Original: {text}\n")
            result_text.insert(tk.END, f"Replaced: {new_text}\n")
            result_text.insert(tk.END, "\n" + "="*80 + "\n")
            result_text.insert(tk.END, "🎯 KEY REGEX CONCEPTS:\n")
            result_text.insert(tk.END, "="*80 + "\n")
            result_text.insert(tk.END, "✓ r'...' - Raw string (essential for backslashes).\n")
            result_text.insert(tk.END, "✓ \\d - Digit (0-9). {n} - Exactly n times.\n")
            result_text.insert(tk.END, "✓ + - One or more times. * - Zero or more times.\n")
            result_text.insert(tk.END, "✓ re.search - Finds first match. re.findall - Finds all matches.\n\n")
        run_btn = tk.Button(parent, text="▶️ Run Regex Demo", command=run_regex_demo)
        style_button(run_btn, bg_color='#3498DB', hover_color='#2980b9')
        run_btn.pack(pady=10)
        run_regex_demo()

    def create_exception_demo(self, parent):
        title = tk.Label(parent, text="🛡️ Exception Handling - Defensive Magic", font=('Segoe UI', 14,
'bold'), fg='#4ECDC4', bg=self.bg_color)
        title.pack(pady=15)
        result_text = scrolledtext.ScrolledText(parent, height=20, width=100, font=('Courier', 10),
bg='#2d3436', fg='#00FF00', wrap=tk.WORD)
        result_text.pack(pady=15, padx=20, fill='both', expand=True)
        def run_exception_demo():
            result_text.delete(1.0, tk.END)
            result_text.insert(tk.END, "🛡️ EXCEPTION HANDLING MAGIC DEMONSTRATION\n")
            result_text.insert(tk.END, "="*80 + "\n\n")
            result_text.insert(tk.END, "📜 Example 1: Handling Division by Zero\n")
            result_text.insert(tk.END, "-" * 80 + "\n")
            code1 = """try:
    result = 10 / 0
except ZeroDivisionError:
    result = "Error: Cannot divide by the grin of the Cheshire Cat!"
"""
            result_text.insert(tk.END, code1 + "\n")
            try:
                result = 10 / 0
            except ZeroDivisionError:
                result = "Error: Cannot divide by the grin of the Cheshire Cat!"
            result_text.insert(tk.END, f"Result: {result}\n")
            result_text.insert(tk.END, "\n📜 Example 2: try/except/else/finally flow\n")
            result_text.insert(tk.END, "-" * 80 + "\n")
            def try_file(filename, fails=False):
                try:
                    if fails:
```

```
                raise FileNotFoundError
            result = f"Successfully processed {filename}"
        except FileNotFoundError:
            result = f"Error: {filename} not found!"
        else:
            result += " (Else block executed)"
        finally:
            result += " (Finally block executed)"
        return result
    result_text.insert(tk.END, try_file("config.txt", fails=False) + "\n")
    result_text.insert(tk.END, try_file("missing.txt", fails=True) + "\n")
    result_text.insert(tk.END, "\n" + "="*80 + "\n")
    result_text.insert(tk.END, "🎯 KEY EXCEPTION CONCEPTS:\n")
    result_text.insert(tk.END, "="*80 + "\n")
    result_text.insert(tk.END, "✓ try: The code that might raise an error.\n")
    result_text.insert(tk.END, "✓ except: The code that runs when a specific error occurs.\n")
    result_text.insert(tk.END, "✓ else: Runs if the 'try' block completes with NO errors.\n")
    result_text.insert(tk.END, "✓ finally: Cleanup code that ALWAYS runs.\n\n")
run_btn = tk.Button(parent, text="▶️ Run Exception Demo", command=run_exception_demo)
style_button(run_btn, bg_color='#3498DB', hover_color='#2980b9')
run_btn.pack(pady=10)
run_exception_demo()

def show_visualization_demo(self):
    if not MATPLOTLIB_AVAILABLE or not PANDAS_AVAILABLE:
        messagebox.showerror("Error", "NumPy, Pandas, or Matplotlib is not installed. Cannot run
demo.")
        return
    viz_window = tk.Toplevel(self.root)
    viz_window.title("📊 Data Visualization - Croquet Ground Analytics")
    viz_window.geometry("900x750")
    viz_window.configure(bg=self.bg_color)
    viz_window.transient(self.root)
    header = tk.Frame(viz_window, bg='#06D6A0')
    header.pack(fill='x')
    tk.Label(header, text="📊 Game Analytics Visualization 📊", font=('Segoe UI', 20, 'bold'),
fg='white', bg='#06D6A0', pady=20).pack()
    try:
        df = self.profile_manager.get_game_statistics()
        if df is None or df.empty:
            data = {
                'realm_name': ['python', 'cpp', 'web', 'data'],
                'average_score': [75.5, 62.0, 88.3, 79.1],
                'attempts_before_full_score': [1.2, 1.8, 1.1, 1.5],
```

```
            'relative_difficulty': [0.245, 0.380, 0.117, 0.209],
            'number_of_players_who_played_realm': [10, 8, 12, 9]
        }
        df = pd.DataFrame(data)
        tk.Label(viz_window, text="Note: Using internal dummy data as game stats are empty.",
fg='#e17055', bg=self.bg_color, font=('Segoe UI', 10)).pack()
    except Exception as e:
        messagebox.showerror("Data Error", f"Could not load/create visualization data: {e}")
        return
    canvas_frame = tk.Frame(viz_window, bg=self.card_color, width=750, height=450)
    canvas_frame.pack(pady=15, padx=20, fill='both', expand=True)
    control_frame = tk.Frame(viz_window, bg=self.bg_color)
    control_frame.pack(pady=10)
    result_text = scrolledtext.ScrolledText(viz_window, height=5, width=100, font=('Courier', 10),
bg='#2d3436', fg='#00FF00', wrap=tk.WORD)
    result_text.pack(pady=5, padx=20, fill='x')
    def run_viz_demo(plot_type):
        for widget in canvas_frame.winfo_children():
            widget.destroy()
        result_text.delete(1.0, tk.END)
        fig, ax = plt.subplots(figsize=(7, 4.5), facecolor=self.bg_color)
        ax.set_facecolor(self.card_color)
        fig.subplots_adjust(bottom=0.25, left=0.15)
        ax.tick_params(axis='x', colors=self.text_dark)
        ax.tick_params(axis='y', colors=self.text_dark)
        ax.yaxis.label.set_color(self.text_dark)
        ax.xaxis.label.set_color(self.text_dark)
        ax.title.set_color(self.primary_color)
        if plot_type == 'bar':
            realms = df['realm_name']
            scores = df['average_score']
            bar_colors = ['#3498DB', '#4ECDC4', '#FFD166', '#06D6A0']
            ax.bar(realms, scores, color=bar_colors)
            ax.set_title('Average Score by Realm', fontsize=14, fontweight='bold')
            ax.set_ylabel('Average Score / 100')
            ax.set_xlabel('Programming Realm')
            for i, score in enumerate(scores):
                ax.text(i, score + 1, f'{score:.1f}', ha='center', color=self.text_dark)
            ax.set_ylim(0, 100)
            result_text.insert(tk.END, "--- Bar Plot Analysis ---\n")
            result_text.insert(tk.END, "Visualizing average Score for each Realm. Higher bars = better
performance.\n")
        elif plot_type == 'line':
            if 'Time_Mins' not in df.columns:
```

```
            df['Time_Mins'] = np.random.randint(5, 30, size=len(df))
        sorted_df = df.sort_values('Time_Mins')
        line_colors = ['#3498DB', '#4ECDC4', '#FFD166', '#06D6A0']
        for idx, (i, row) in enumerate(sorted_df.iterrows()):
            color = line_colors[idx % len(line_colors)]
            ax.plot([0, row['Time_Mins']], [0, row['average_score']], marker='o', markersize=10,
color=color, label=row['realm_name'], linewidth=2)
        ax.set_title('Score vs Time Performance by Realm', fontsize=14, fontweight='bold')
        ax.set_xlabel('Mock Average Time Spent (Mins)')
        ax.set_ylabel('Average Score Achieved')
        ax.legend(loc='best', frameon=True, fancybox=True, shadow=True)
        ax.grid(True, linestyle='--', alpha=0.3)
        result_text.insert(tk.END, "\n--- Line Chart Analysis ---\n")
        result_text.insert(tk.END, "Shows performance trends over time. Each colored line represents a
different realm.\n")
    canvas = FigureCanvasTkAgg(fig, master=canvas_frame)
    canvas.draw()
    canvas.get_tk_widget().pack(fill='both', expand=True)
bar_btn = tk.Button(control_frame, text="📊 Bar Plot (Avg. Scores)", command=lambda:
run_viz_demo('bar'))
style_button(bar_btn, bg_color='#06D6A0', hover_color='#019874')
bar_btn.pack(side='left', padx=10, pady=5)
line_btn = tk.Button(control_frame, text="📈 Line Chart (Score vs Time)", command=lambda:
run_viz_demo('line'))
style_button(line_btn, bg_color='#3498DB', hover_color='#2980b9')
line_btn.pack(side='left', padx=10, pady=5)
run_viz_demo('bar')
self.center_window(viz_window)

def show_profile_dashboard(self):
    profile = self.profile_manager.profiles.get(self.player_name, {})
    stats = self.profile_manager.get_player_statistics(self.player_name)
    dashboard_window = tk.Toplevel(self.root)
    dashboard_window.title(f"👤 {self.player_name}'s Adventurer Dashboard")
    dashboard_window.geometry("1000x750")
    dashboard_window.configure(bg=self.bg_color)
    dashboard_window.transient(self.root)
    header = tk.Frame(dashboard_window, bg=self.primary_color)
    header.pack(fill='x')
    tk.Label(header, text=f"✨ {self.player_name}'s Journey ✨", font=('Segoe UI', 24, 'bold'),
fg='white', bg=self.primary_color, pady=20).pack()
    content_frame = tk.Frame(dashboard_window, bg=self.bg_color)
    content_frame.pack(fill='both', expand=True, padx=20, pady=20)
    left_panel = tk.Frame(content_frame, bg=self.bg_color, width=400)
```

```
        left_panel.pack(side='left', fill='both', padx=(0, 10))
        right_panel = tk.Frame(content_frame, bg=self.bg_color)
        right_panel.pack(side='right', fill='both', expand=True, padx=(10, 0))
        stats_card = tk.Frame(left_panel, bg=self.card_color, relief='flat')
        stats_card.pack(fill='both', expand=True)
        tk.Label(stats_card, text="📊 Adventure Statistics", font=('Segoe UI', 16, 'bold'),
fg=self.primary_color, bg=self.card_color, pady=15).pack()
        stats_list = [
            ("🏆 Total Magic Score:", stats.get('total_score', 0)),
            ("🏰 Realms Conquered:", stats.get('realms_completed', 0)),
            ("⏱️ Total Playtime (s):", profile.get('total_playtime', 0)),
            ("📈 Last Replay Diff:", stats.get('last_replay_difference', 0)),
            ("⭐ Favorite Level:", stats.get('favorite_realm_level', 'N/A')),
            ("📅 Profile Created:", datetime.strptime(stats.get('created_date', ''), '%Y-%m-%d
%H:%M:%S').strftime('%b %d, %Y') if stats.get('created_date') else 'N/A'),
        ]
        for label, value in stats_list:
            stat_frame = tk.Frame(stats_card, bg=self.card_color)
            stat_frame.pack(fill='x', padx=15, pady=5)
            tk.Label(stat_frame, text=label, font=('Segoe UI', 11), fg=self.text_dark, bg=self.card_color,
anchor='w').pack(side='left')
            tk.Label(stat_frame, text=str(value), font=('Segoe UI', 11, 'bold'), fg=self.primary_color,
bg=self.card_color, anchor='e').pack(side='right')
        tk.Label(stats_card, text="🎮 Realm Attempts:", font=('Segoe UI', 13, 'bold'),
fg=self.secondary_color, bg=self.card_color, pady=10).pack()
        realm_play_counts = stats.get('realm_play_counts', {})
        if realm_play_counts:
            for realm, count in realm_play_counts.items():
                realm_name = self.realms.get(realm, {}).get('name', realm)
                tk.Label(stats_card, text=f"  • {realm_name}: {count} attempts", font=('Segoe UI', 10),
fg=self.text_dark, bg=self.card_color, anchor='w').pack(fill='x', padx=20)
        else:
            tk.Label(stats_card, text="  No realm attempts recorded yet", font=('Segoe UI', 10), fg='#636e72',
bg=self.card_color, anchor='w').pack(fill='x', padx=20)
        tk.Label(stats_card, text="📊 Difficulty Performance:", font=('Segoe UI', 13, 'bold'),
fg=self.secondary_color, bg=self.card_color, pady=10).pack()
        realm_diff_stats = stats.get('realm_difficulty_stats', {})
        if realm_diff_stats:
            for realm_k, diff_stats in realm_diff_stats.items():
                realm_name = self.realms.get(realm_k, {}).get('name', realm_k)
                tk.Label(stats_card, text=f"→ {realm_name}", font=('Segoe UI', 11, 'underline'),
fg=self.text_dark, bg=self.card_color, anchor='w').pack(fill='x', padx=20, pady=(5, 0))
                for diff_k, data in diff_stats.items():
                    scores = data.get('scores', [])
```

```
            attempts = data.get('attempts', 0)
            avg = sum(scores) / len(scores) if scores else 0
            tk.Label(stats_card, text=f"  • {diff_k.title()}: Avg Score: {avg:.1f} | Attempts: {attempts}",
font=('Segoe UI', 9), fg='#636e72', bg=self.card_color, anchor='w').pack(fill='x', padx=30)
        else:
            tk.Label(stats_card, text="  No difficulty statistics recorded yet", font=('Segoe UI', 10),
fg='#636e72', bg=self.card_color, anchor='w').pack(fill='x', padx=20)
        viz_card = tk.Frame(right_panel, bg=self.card_color, relief='flat')
        viz_card.pack(fill='both', expand=True)
        tk.Label(viz_card, text="🎯 Realm Completion Progress", font=('Segoe UI', 16, 'bold'),
fg=self.primary_color, bg=self.card_color, pady=15).pack()
        if MATPLOTLIB_AVAILABLE:
            canvas_frame = tk.Frame(viz_card, bg=self.card_color, width=450, height=450)
            canvas_frame.pack(fill='both', expand=True, padx=10, pady=10)
            completed = len(self.completed_realms)
            total = len(self.realms)
            remaining = total - completed
            fig, ax = plt.subplots(figsize=(5, 5), facecolor=self.card_color)
            if completed == 0 and remaining > 0:
                labels = ['Remaining']
                data = [remaining]
                colors = ['#e17055']
                autopct = None
            elif completed > 0 and remaining == 0:
                labels = ['Completed']
                data = [completed]
                colors = ['#00b894']
                autopct = '%1.1f%%'
            else:
                labels = ['Completed', 'Remaining']
                data = [completed, remaining]
                colors = ['#00b894', '#e17055']
                autopct = '%1.1f%%'
            wedges, texts, autotexts = ax.pie(data, labels=labels, autopct=autopct, colors=colors,
startangle=90, shadow=True, textprops={'fontsize': 12, 'weight': 'bold'})
            for text in texts:
                text.set_color(self.text_dark)
            if autotexts:
                for autotext in autotexts:
                    autotext.set_color('white')
            ax.set_title('Realm Completion Progress', fontsize=13, color=self.primary_color, weight='bold')
            ax.axis('equal')
            canvas = FigureCanvasTkAgg(fig, master=canvas_frame)
            canvas.draw()
```

```
            canvas.get_tk_widget().pack(fill='both', expand=True)
        else:
            tk.Label(viz_card, text="Matplotlib is not installed.\nCannot display chart.", font=('Segoe UI', 12),
fg='#e17055', bg=self.card_color).pack(pady=50)
        self.center_window(dashboard_window)

if __name__ == '__main__':
    root = tk.Tk()
    profile_manager = ProfileManager()
    def start_alice_app(player_name):
        for widget in root.winfo_children():
            if isinstance(widget, tk.Frame) and widget.winfo_ismapped():
                widget.destroy()
        Alice4App(root, player_name, profile_manager)
    ProfileSelectScreen(root, profile_manager, start_alice_app)
    root.mainloop()
```

CHAPTER 7: OUTPUT SCREENSHOTS

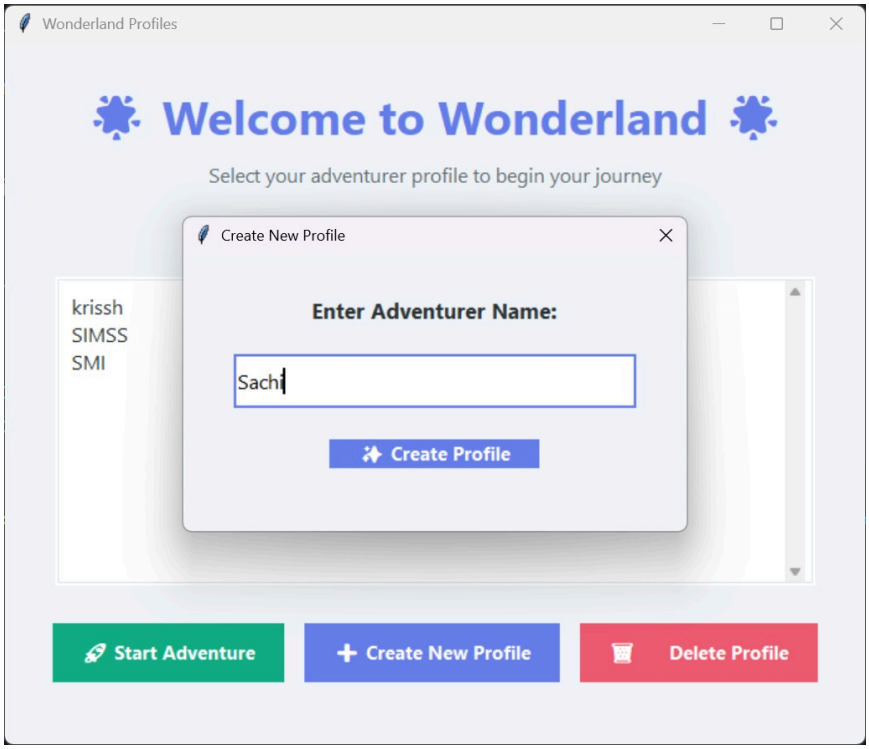

Fig. 7.1 Depicting Creation of Profile

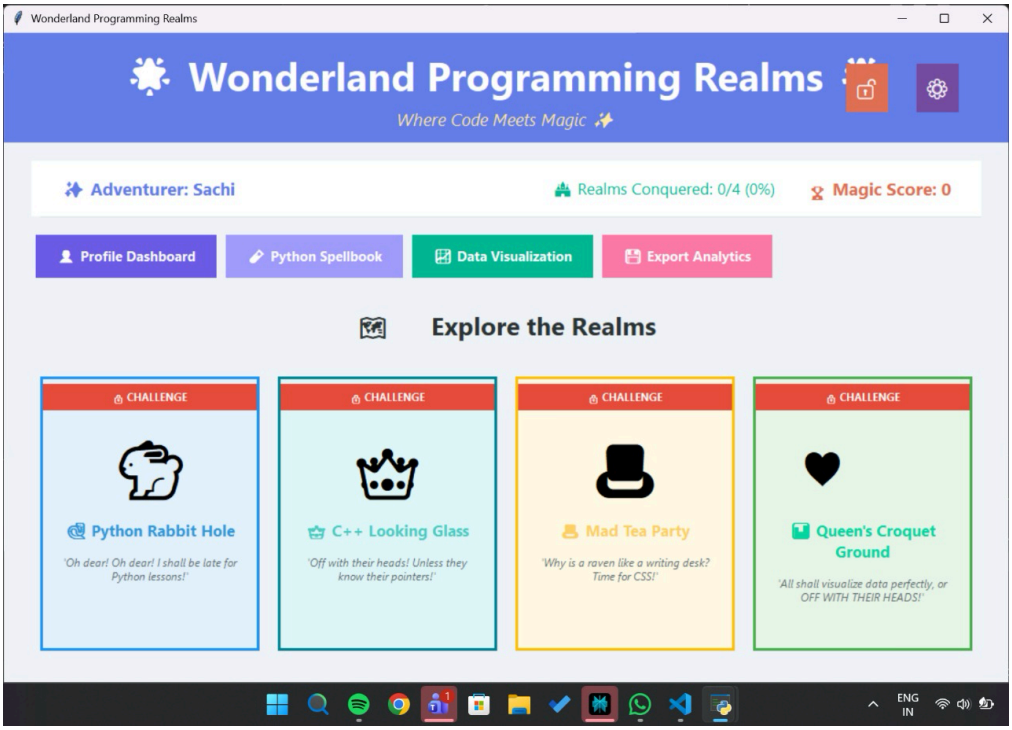

Fig. 7.2. Depicting Welcome Page

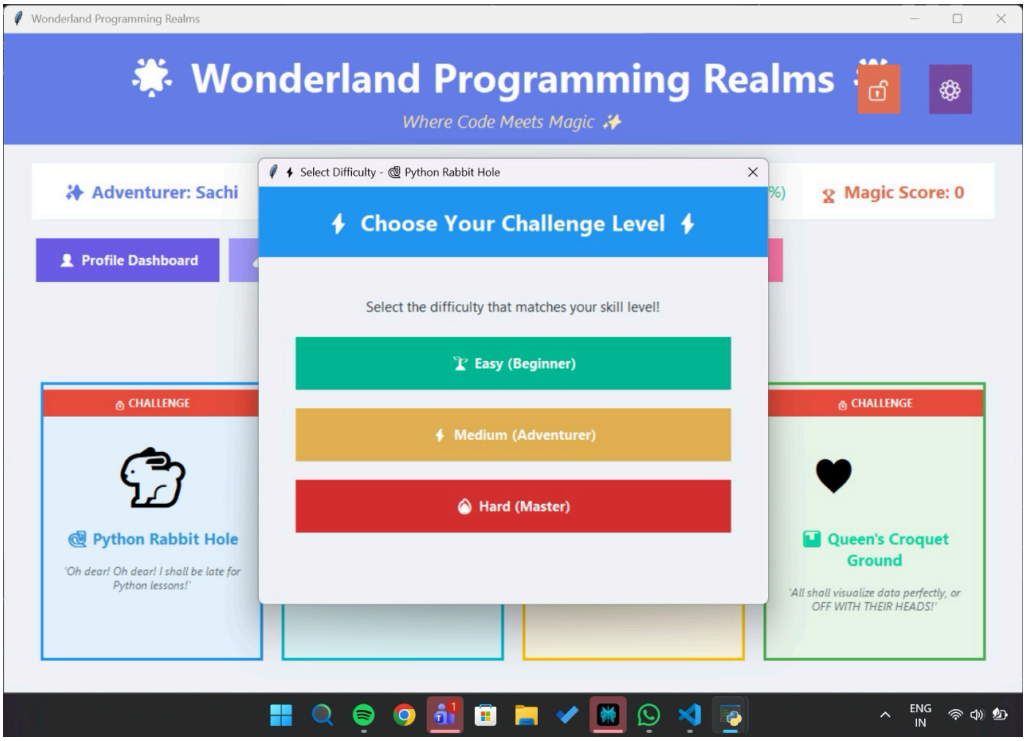

Fig. 7.3. Displaying different holes in the Python Rabbit hole

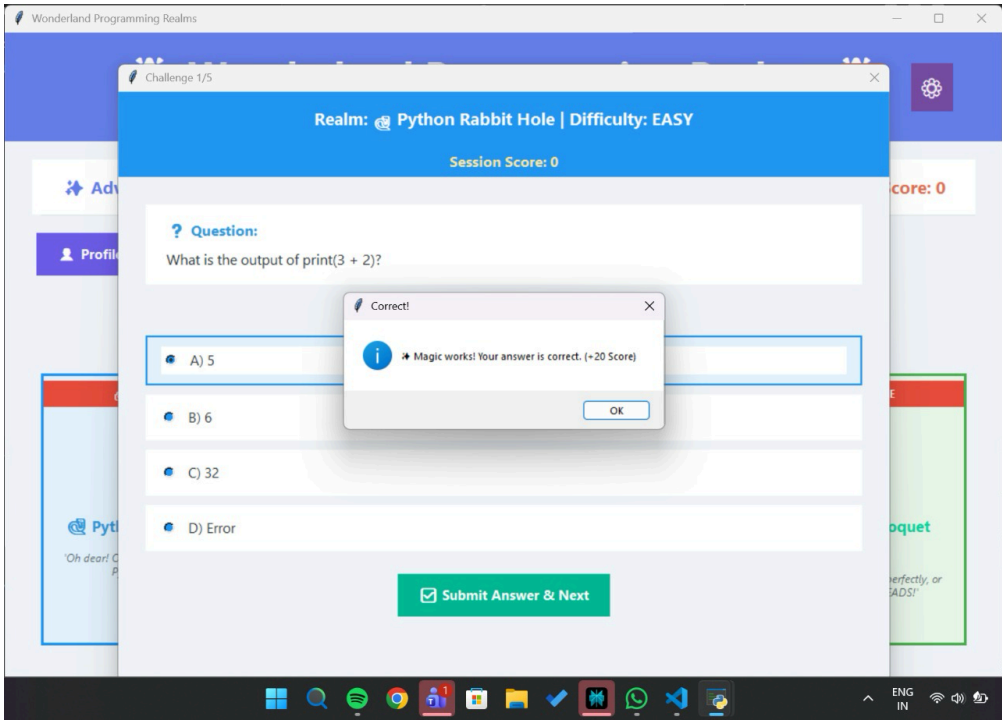

Fig. 7.4. Depicting Challenge One in the Python Rabbit Hole Realm

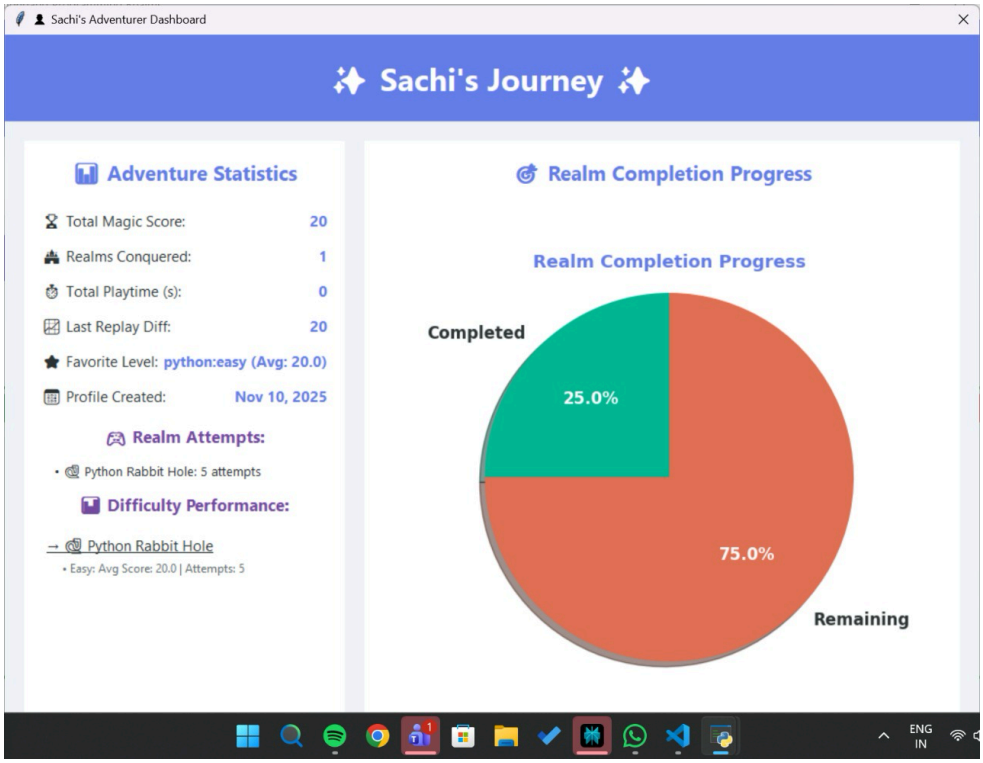

Fig. 7.5. Depicting Adventurer's Dashboard

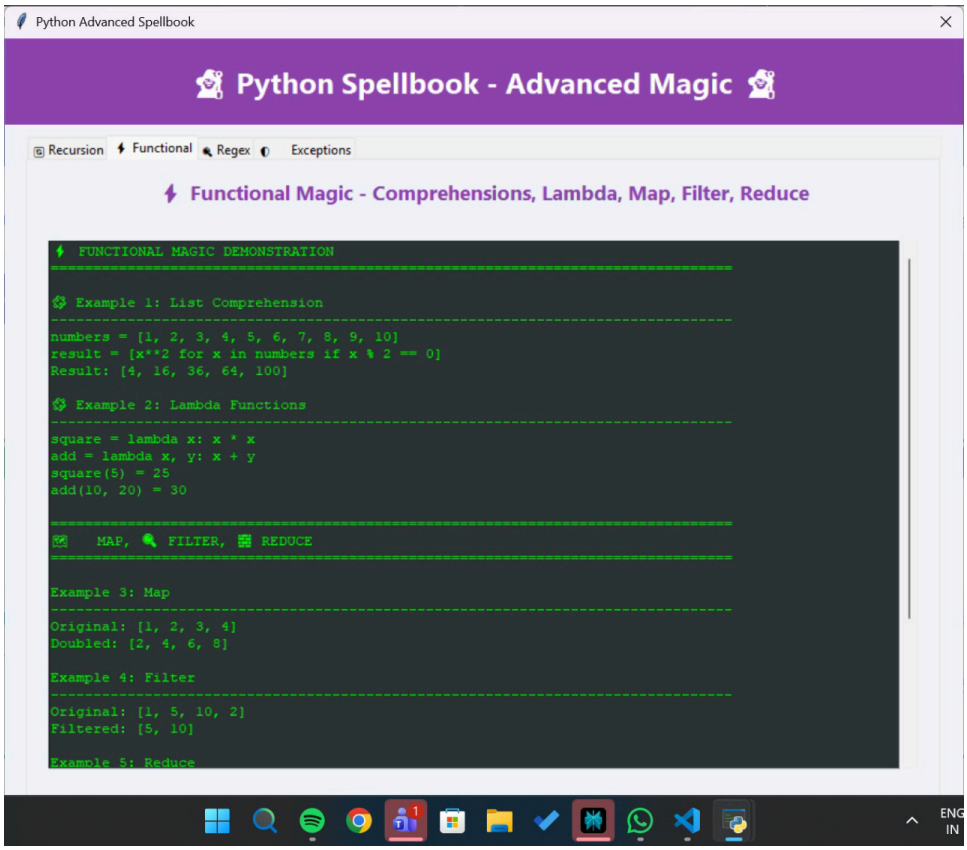

Fig. 7.6. Python Spellbook

CHAPTER 8: SCOPE OF IMPROVEMENT

"Wonderland Programming Realms", in its current state, establishes an effective and engaging base. It can be much improved in terms of scope, scalability, and overall user experience through the following possible improvements:

1. Content & Pedagogical Depth
   - Varied Challenge Formats: Creating challenges that are not limited to multiple-choice questions, and offer a wider range of interactive challenges such as: completing code snippets, debugging challenges, fill-in-the-blank code challenges, and even basic output prediction would allow for a much wider celebration of assessment of a user's understanding.
   - Embedded code editor and live execution capabilities: This is the greatest pedagogical advancement, in my opinion. Incorporating a simple code editor inside the challenge windows would be incredible. It would be a game-changer if users could write code and execute small snippets of code (in a secure sandboxed way) with live output to the app itself. This would elevate the app from a question/quizzer to an actual interactive learning platform.

2. Technical Architecture and Performance

- Modular Codebase with MVC Pattern: While the existing code is technically object-oriented, it combines UI logic (Tkinter commands) with application/business logic. With a strict adherence to the Model-View-Controller (MVC) pattern, future development would be improved in terms of maintainability, testability, and ease in swapping the UI layer (e.g., moving to a web-based frontend in the future).

3. User Experience (UX) & Gamification Satisfaction

- More Sophisticated Gamification Elements: Adding experience points (XP) separate from the score, unlockable avatars or themes, daily login streaks, and a global leaderboard would increase user engagement and replayability in the long run.
- Better Detailed Progress Analytics: Rather than just getting the score, a dedicated "Stats" screen could show users their strengths and weaknesses—e.g., "You are 90% accurate on Python loops but only 60% on C++ pointers," to help guide them in their revision and give additional context to their own learning experience.

4. Features & Deployment
   - Administrator Panel: Allowing educators or administrators to create a dedicated mode to add, edit, or delete questions and realms without changing the source code would enhance versatility and usability in the classroom.
   - Cross-Platform Compatibility & Packaging: Although Python and Tkinter are cross-platform, formally packaging the application into standalone executables for Windows, Mac, and Linux (using PyInstaller or cx_Freeze) would significantly reduce the barrier to entry for end-users who aren't familiar with running Python scripts.
   - Cloud Synchronization: A future-looking feature would be to offer optional cloud storage for user profiles, enabling learners to easily continue their progress on different machines.

CHAPTER 9: REFERENCES

1. Chun, W. J. (2006). *Core Python Programming* (2nd ed.). Prentice Hall. Retrieved from https://theswissbay.ch/pdf/Gentoomen%20Library/Programming/Python/Core%20Python%20Programming%2C%20Second%20Edition%20%282006%29.pdf
2. Chen, D. Y. (2018). *Pandas for Everyone: Python Data Analysis* (1st ed.). Addison-Wesley Professional. Retrieved from https://ptgmedia.pearsoncmg.com/images/9780134546933/samplepages/9780134546933_Sample.pdf

3. Amos, D., Bader, D., Jablonski, J., & Heisler, F. (2020). *Python Basics: A Practical Introduction to Python 3* (4th ed.). Real Python. Retrieved from https://static.realpython.com/python-basics-sample-chapters.pdf
4. McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media. https://books.google.com/books/about/Python_for_Data_Analysis.html?id=EAKBEAAAQBAJ